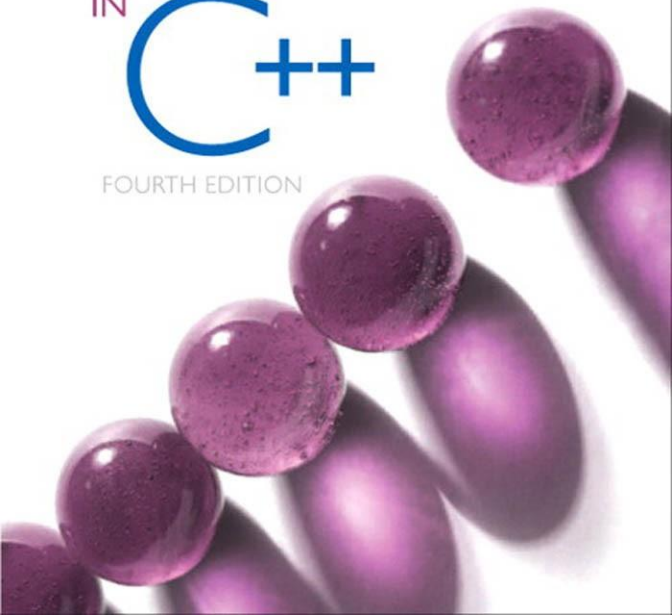




MARK ALLEN WEISS

DATA STRUCTURES
AND
ALGORITHM ANALYSIS
IN
C++

FOURTH EDITION



Edisi Keempat

Struktur Data dan Algoritma Analisis di C++

Edisi Keempat

Struktur Data dan Algoritma

Analisis di

C++

Mark Allen Weiss

Universitas Internasional Florida

PEARSON

Boston Columbus Indianapolis New York San Francisco
Upper Saddle River Amsterdam Cape Town Dubai London
Madrid Milan Munich Paris Montreal Toronto Delhi Mexico
City Sao Paulo Sydney Hong Kong Seoul Singapura Taipei
Tokyo

List, Stack, and Queues

Bab ini membahas tiga struktur data paling sederhana dan mendasar. Hampir setiap program penting akan menggunakan setidaknya satu dari struktur ini secara eksplisit, dan tumpukan selalu digunakan secara implisit dalam suatu program, terlepas dari apakah Anda mendeklarasikannya atau tidak. Di antara poin-poin penting dalam bab ini, kita akan...

- Memperkenalkan konsep Tipe Data Abstrak (ADT).
- Tunjukkan cara melakukan operasi pada daftar secara efisien.
- Perkenalkan tumpukan ADT dan penggunaannya dalam mengimplementasikan rekursi.
- Perkenalkan antrean ADT dan penggunaannya dalam sistem operasi dan desain algoritma.

Dalam bab ini, kami menyediakan kode yang mengimplementasikan subset penting dari dua kelas pustaka: vektor dan daftar.

3.1 Tipe Data Abstrak (ADT)

Sebuah tipe data abstrak (ADT) adalah sekumpulan objek beserta sekumpulan operasinya. Tipe data abstrak adalah abstraksi matematika; tidak ada satu pun definisi ADT yang menyebutkan bagaimana himpunan operasi diimplementasikan. Objek seperti daftar, himpunan, dan grafik, beserta operasinya, dapat dipandang sebagai ADT, sebagaimana bilangan bulat, bilangan riil, dan boolean merupakan tipe data. Bilangan bulat, bilangan riil, dan boolean memiliki operasi yang terkait dengannya, begitu pula ADT. Untuk ADT himpunan, kita mungkin memiliki operasi seperti menambah, menghapus, ukuran, dan berisi. Atau, kita mungkin hanya menginginkan dua operasi union dan find, yang akan mendefinisikan ADT yang berbeda pada set tersebut.

Kelas C++ memungkinkan implementasi ADT, dengan menyembunyikan detail implementasi yang sesuai. Dengan demikian, bagian lain dari program yang perlu melakukan operasi pada ADT dapat melakukannya dengan memanggil metode yang sesuai. Jika karena suatu alasan detail implementasi perlu diubah, seharusnya mudah dilakukan hanya dengan mengubah rutin yang menjalankan operasi ADT. Perubahan ini, dalam dunia yang ideal, akan sepenuhnya transparan bagi keseluruhan program.

Tidak ada aturan yang memberi tahu kita operasi mana yang harus didukung untuk setiap ADT; ini adalah keputusan desain. Penanganan kesalahan dan pemecahan masalah (jika diperlukan) juga umumnya bergantung pada perancang program. Tiga struktur data yang akan kita pelajari dalam bab ini adalah contoh utama ADT. Kita akan melihat bagaimana masing-masing dapat diimplementasikan dengan beberapa cara, tetapi

jika dilakukan dengan benar, program yang menggunakannya tidak perlu mengetahui implementasi mana yang digunakan.

3.2 Daftar ADT

Kami akan membahas daftar umum formulirnya $A_0, A_1, A_2, \dots, A_{N-1}$. Kami mengatakan bahwa ukuran daftar ini adalah N . Kita akan menyebut daftar khusus berukuran 0 sebagai **daftar kosong**.

Untuk daftar apa pun kecuali daftar kosong, kita katakan bahwa A_{i-1} mengikuti (atau berhasil) A_{i-1} ($i < N$) dan itu A_{i-1} mendahului A_i ($i > 0$). Elemen pertama dari daftar tersebut adalah A_0 , dan elemen terakhir adalah A_{N-1} . Kami tidak akan mendefinisikan pendahulu dari A_0 atau penerusnya A_{N-1} . Itu posisi dari elemen A_i dalam sebuah daftar adalah i . Sepanjang diskusi ini, untuk menyederhanakan masalah, kami akan berasumsi bahwa elemen-elemen dalam daftar adalah bilangan bulat, tetapi secara umum, elemen-elemen dengan kompleksitas yang sewenang-wenang diperbolehkan (dan mudah ditangani oleh templat kelas).

Terkait dengan “definisi” ini adalah serangkaian operasi yang ingin kami lakukan pada Daftar ADT. Beberapa operasi yang populer adalah *printList* dan *makeEmpty*, yang melakukan hal-hal yang jelas; *find*, yang mengembalikan posisi kemunculan pertama suatu item; *insert* dan *remove*, yang umumnya memasukkan dan menghapus beberapa elemen dari beberapa posisi dalam daftar; dan *findhKth*, yang mengembalikan elemen pada posisi tertentu (ditentukan sebagai argumen). Jika daftarnya adalah 34, 12, 52, 16, 12, maka *findh(52)* mungkin mengembalikan 2; *insert(x, 2)* mungkin membuat daftar menjadi 34, 12, X, 52, 16, 12 (jika kita masukkan ke posisi yang diberikan); dan *remove(52)* mungkin mengubah daftar itu menjadi 34, 12, X, 16, 12.

Tentu saja, penafsiran tentang apa yang tepat untuk suatu fungsi sepenuhnya tergantung pada programmer, seperti halnya penanganan kasus khusus (misalnya, apa yang temukan(1) kembali di atas?). Kita juga bisa menambahkan operasi seperti *next* dan *previous*, yang akan mengambil posisi sebagai argumen dan mengembalikan posisi penerus dan pendahulu, masing-masing.

3.2.1 Implementasi Array Sederhana dari Daftar

Semua instruksi ini dapat diimplementasikan hanya dengan menggunakan array. Meskipun array dibuat dengan kapasitas tetap, vektor Kelas, yang menyimpan array secara internal, memungkinkan array tersebut berkembang dengan menggandakan kapasitasnya saat dibutuhkan. Ini memecahkan masalah paling serius dalam penggunaan array—yaitu, secara historis, untuk menggunakan array, diperlukan estimasi ukuran maksimum daftar. Estimasi ini tidak lagi diperlukan.

Implementasi array memungkinkan daftar cetak untuk dilakukan dalam waktu linier, dan *findhKth* Operasi ini membutuhkan waktu yang konstan, yang merupakan hal terbaik yang dapat diharapkan. Namun, penyisipan dan penghapusan berpotensi mahal, tergantung di mana penyisipan dan penghapusan terjadi. Dalam kasus terburuk, penyisipan ke posisi 0 (dengan kata lain, di awal daftar) mengharuskan seluruh array didorong ke bawah satu titik untuk memberi ruang, dan menghapus elemen pertama mengharuskan semua elemen dalam daftar digeser ke atas satu titik. Jadi, kasus terburuk untuk operasi ini adalah $O(N)$. Rata-rata, setengah dari daftar perlu dipindahkan untuk kedua operasi tersebut, sehingga waktu linier tetap diperlukan. Di sisi lain, jika semua operasi terjadi di ujung atas daftar, maka tidak ada elemen yang perlu digeser, dan kemudian penambahan dan penghapusan membutuhkan waktu $O(1)$ waktu.

Ada banyak situasi di mana daftar dibangun dengan penyisipan di ujung atas, dan kemudian hanya akses array (yaitu, *findhKth* operasi) terjadi. Dalam kasus seperti itu, array merupakan implementasi yang sesuai. Namun, jika penyisipan dan penghapusan terjadi di seluruh daftar dan, khususnya, di bagian depan daftar, maka array bukanlah pilihan yang baik. Bagian selanjutnya membahas alternatifnya: *daftar tertaut*.

3.2.2 Daftar Tertaut Sederhana

Untuk menghindari biaya linear penyisipan dan penghapusan, kita perlu memastikan bahwa daftar tersebut tidak disimpan secara berurutan, karena jika tidak, seluruh bagian daftar perlu dipindahkan. Gambar 3.1 menunjukkan gambaran umum daftar tertaut.

Daftar tertaut terdiri dari serangkaian simpul, yang tidak selalu berdekatan dalam memori. Setiap simpul berisi elemen dan tautan ke simpul yang berisi penerusnya. Kami menyebutnya Berikutnya tautan. Sel terakhir Berikutnya titik tautan *nullptr*.

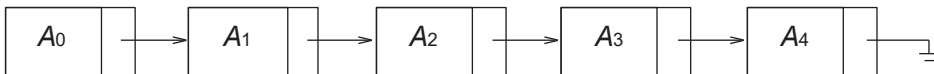
Untuk mengeksekusi *printhlist()* atau *find(x)*, kita hanya memulai pada node pertama dalam daftar dan kemudian melintasi daftar dengan mengikuti berikutnya tautan. Operasi ini jelas bersifat linear-waktu, seperti pada implementasi array; *findhKth*, konstanta kemungkinan akan lebih besar daripada jika implementasi array digunakan. *findhKth* operasi tidak lagi seefisien implementasi array; *findhKth(i)* mengambil $O(i)$ waktu dan bekerja dengan menelusuri daftar dengan cara yang jelas. Dalam praktiknya, batasan ini pesimistis, karena sering kali panggilan ke *findhKth* diurutkan berdasarkan urutan (by *i*). Sebagai contoh, *findhKth* (2), *findhKth* (3), *findhKth* (4), Dan *findhKth* (6) semuanya dapat dieksekusi dalam satu pemindaian pada daftar.

Itu menghapus metode dapat dieksekusi dalam satu berikutnya perubahan penunjuk. Gambar 3.2 menunjukkan hasil penghapusan elemen ketiga dalam daftar asli.

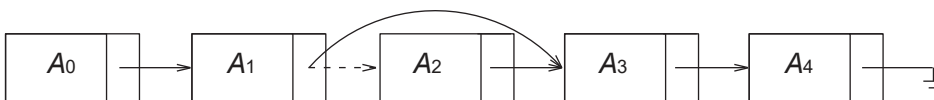
Itu menyisipkan metode ini memerlukan perolehan node baru dari sistem dengan menggunakan baru panggilan dan kemudian mengeksekusi dua berikutnya manuver penunjuk. Ide umumnya ditunjukkan pada Gambar 3.3. Garis putus-putus mewakili penunjuk lama.

Seperti dapat kita lihat, pada prinsipnya, jika kita tahu di mana perubahan harus dilakukan, memasukkan atau menghapus item dari daftar tertaut tidak memerlukan pemindahan banyak item, dan sebaliknya hanya melibatkan jumlah perubahan yang konstan pada tautan simpul.

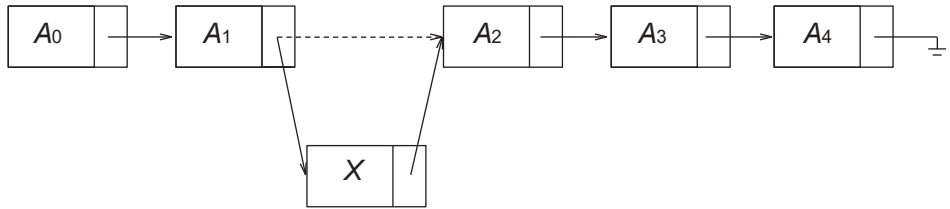
Kasus khusus penambahan ke depan atau penghapusan item pertama dengan demikian merupakan operasi waktu konstan, tentu saja dengan asumsi bahwa tautan ke depan daftar tertaut dipertahankan.



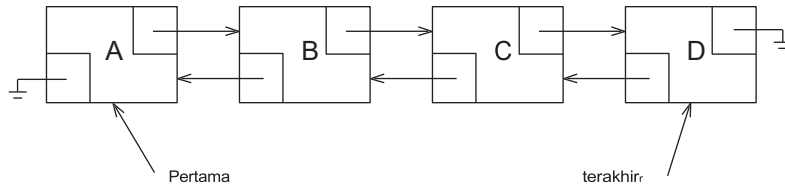
Gambar 3.1 Daftar tertaut



Gambar 3.2 Penghapusan dari daftar tertaut



Gambar 3.3 Penyisipan ke dalam linked list



Gambar 3.4 Daftar tertaut ganda

Kasus khusus penambahan di akhir (yaitu, menjadikan item baru sebagai item terakhir) dapat bersifat konstan-waktu, selama kita mempertahankan tautan ke simpul terakhir. Dengan demikian, daftar tertaut yang umum mempertahankan tautan ke kedua ujung daftar. Menghapus item terakhir lebih rumit, karena kita harus menemukan item kedua terakhir, mengubah berikutnya tautan *nullptr*, lalu perbarui tautan yang mempertahankan simpul terakhir. Dalam daftar tertaut klasik, di mana setiap simpul menyimpan tautan ke simpul berikutnya, keberadaan tautan ke simpul terakhir tidak memberikan informasi apa pun tentang simpul berikutnya.

Gagasan yang jelas untuk mempertahankan tautan ketiga ke simpul kedua terakhir tidak berhasil, karena tautan tersebut juga perlu diperbarui selama penghapusan. Sebagai gantinya, kita meminta setiap simpul untuk mempertahankan tautan ke simpul sebelumnya dalam daftar. Hal ini ditunjukkan pada Gambar 3.4 dan dikenal sebagai daftar tertaut ganda.

3.3 vector dan daftar di STL

Bahasa C++ memiliki implementasi struktur data umum di dalam pustakanya. Bagian bahasa ini dikenal sebagai Pustaka Templat Standar (STL). ADT Daftar adalah salah satu struktur data yang diimplementasikan dalam STL. Kita akan membahas beberapa struktur data lainnya di Bab 4 dan 5. Secara umum, struktur data ini disebut koleksi atau kontainer.

Ada dua implementasi populer dari List ADT. vektor menyediakan implementasi array yang dapat tumbuh dari List ADT. Keuntungan menggunakan vektor adalah bahwa hal itu dapat diindeks dalam waktu yang konstan. Kerugiannya adalah bahwa penyisipan item baru dan penghapusan item yang sudah ada mahal, kecuali perubahan dilakukan pada akhir vektor. Itu daftar menyediakan implementasi daftar tertaut ganda dari List ADT. Keuntungan menggunakan

daftar adalah bahwa penyisipan item baru dan penghapusan item yang sudah ada murah, asalkan posisi perubahannya diketahui. Kerugiannya adalah daftar tidak mudah diindeks. Keduanya vektor dan daftar tidak efisien untuk pencarian. Sepanjang diskusi ini, daftar merujuk kepada daftar tertaut ganda dalam STL, sedangkan daftar (yang disusun tanpa font monospace) merujuk kepada ADT Daftar yang lebih umum.

Keduanya vektor dan daftar adalah templat kelas yang diinstansiasi dengan jenis item yang disimpannya

- `int size ()` konst: mengembalikan jumlah elemen dalam wadah.
- `void clear ()`: menghapus semua elemen dari wadah.
- `bool empty ()` const: mengembalikan true jika wadah tidak berisi elemen apa pun, dan false jika tidak.

Keduanya vektor dan daftar mendukung penambahan dan penghapusan dari akhir daftar dalam waktu yang konstan. Keduanya vektor dan daftar mendukung akses ke item terdepan dalam daftar secara konstan. Operasinya adalah:

- `void push_back ( const Objek & x )`: menambahkan X sampai akhir daftar.
- `void pop_back ()`: menghapus objek pada akhir daftar.
- `const Objek & back ()` const: mengembalikan objek di akhir daftar (mutator yang mengembalikan referensi juga disediakan).
- `const Objek & front ()` const: mengembalikan objek di depan daftar (mutator yang mengembalikan referensi juga disediakan).

Karena daftar tertaut ganda memungkinkan perubahan yang efisien di bagian depan, tetapi vektor tidak, dua metode berikut hanya tersedia untuk daftar:

- `void push_front ( const Objek & x )`: menambahkan X ke depan daftar.
- `void pop_depan ()`: menghapus objek di depan daftar.

Vektor memiliki serangkaian metode khusus yang tidak termasuk dalam daftar metode standar. Dua metode digunakan untuk melakukan pengindeksan secara efisien, sementara dua metode lainnya memungkinkan programmer untuk melihat serta mengubah kapasitas internal. Metode-metode tersebut adalah:

- `Objek & operator[] ( int idx )`: mengembalikan objek pada indeks idx di dalam vektor, tanpa pemeriksaan batas (aksesor yang mengembalikan referensi konstan juga disediakan).
- `Objek & at ( int idx )`: mengembalikan objek pada indeks idx di dalam vektor, dengan boundschecking (aksesor yang mengembalikan referensi konstan juga disediakan).
- `int capacity ()` const: mengembalikan kapasitas internal vektor (Lihat Bagian 1.4 untuk rincian lebih lanjut.)
- `void reserve ( int newCapacity )`: menetapkan kapasitas baru. Jika perkiraan yang baik tersedia, perkiraan tersebut dapat digunakan untuk menghindari perluasan vektor. (Lihat Bagian 1.4 untuk rincian lebih lanjut.)

3.3.1 Iterator

Beberapa operasi pada daftar, terutama operasi penyisipan dan penghapusan dari tengah daftar, memerlukan konsep posisi. Dalam STL, posisi direpresentasikan oleh tipe bersarang, pengulangan. Khususnya, untuk daftar<string>, posisi diwakili oleh tipe `list<string>::iterator`; untuk sebuah `vector<int>`, posisi diwakili oleh kelas `vector<int>::iterator`, dan seterusnya. Dalam menjelaskan beberapa metode, kita hanya akan menggunakan pengulangan sebagai singkatan, tetapi saat menulis kode, kita akan menggunakan nama kelas bersarang yang sebenarnya.

Pada awalnya, ada tiga isu utama yang harus dibahas: pertama, bagaimana seseorang mendapatkan iterator; kedua, operasi apa yang dapat dilakukan oleh iterator itu sendiri; ketiga, metode ADT Daftar mana yang memerlukan iterator sebagai parameter.

Mendapatkan Iterator

Untuk masalah pertama, daftar STL (dan semua wadah STL lainnya) mendefinisikan sepasang metode:

- `iterator begin ( )`: mengembalikan iterator yang sesuai yang mewakili item pertama dalam wadah.
- `iterator akhir ( )`: mengembalikan iterator yang sesuai yang mewakili penanda akhir dalam wadah (yaitu, posisi setelah item terakhir dalam wadah).

pertimbangkan kode berikut yang biasanya digunakan untuk mencetak item dalam vektor sebelum diperkenalkannya berbasis jangkauan untuk loops dalam C++11:

```
for ( int i = 0; i != v.size( ); ++i )
    cout << v[ i ] << endl;
```

Jika kita menulis ulang kode ini menggunakan iterator, kita akan melihat korespondensi alami dengan `mulai` Dan `akhir` metode:

```
for( vector<int>::iterator itr = v.begin( ); itr != v.end( );
    itr.??? ) cout << itr.??? << endl;
```

Dalam pengujian terminasi loop, keduanya `itr != v.size( )` dan `itr != v.end ( )` dimaksudkan untuk menguji apakah penghitung loop telah menjadi “di luar batas.” Fragmen kode juga membawa kita ke masalah kedua, yaitu bahwa iterator harus memiliki metode yang terkait dengannya (metode yang tidak diketahui ini diwakili oleh ???).

Metode Iterator

Berdasarkan fragmen kode di atas, jelas bahwa iterator dapat dibandingkan dengan `!=` Dan `==`, dan mungkin memiliki konstruktor salinan dan operator `=` didefinisikan. Dengan demikian, iterator memiliki metode, dan banyak metode menggunakan operator overloading. Selain menyalin, operasi yang paling umum digunakan pada iterator meliputi:

- `itr++` and `++itr` : memajukan iterator itu ke lokasi berikutnya. Baik bentuk awalan maupun akhiran diperbolehkan.

- itr: mengembalikan referensi ke objek yang disimpan di iterator ituLokasi. Referensi yang dikembalikan mungkin dapat diubah atau tidak (detail ini akan segera kami bahas).

- `itr1==itr2`: mengembalikan true jika iterator itr1 Dan itr2 merujuk ke lokasi yang sama dan salah jika tidak
- `itr1!=itr2`: mengembalikan true jika iterator itr1 Dan itr2 merujuk ke lokasi berbeda dan salah jika tidak.

Dengan operator ini, kode yang akan dicetak adalah

```
for (vector<int>::iterator itr = v.begin(); itr !=v.end(); ++itr)
    cout << *itr << endl;
```

Penggunaan operator overloading memungkinkan seseorang untuk mengakses item saat ini, lalu maju ke item berikutnya menggunakan `*itr++`. Oleh karena itu, alternatif dari fragmen di atas adalah

```
vector<int>::iterator itr = v.begin();
sementara( itr !=v.akhir() )
    cout << *itr++ << endl;
```

Operasi Kontainer yang Memerlukan Iterator

Untuk masalah terakhir, tiga metode paling populer yang memerlukan iterator adalah metode yang menambah atau menghapus dari daftar (baik vektor atau daftar) pada posisi tertentu:

- iterator insert (iterator pos, const Objek & x): menambahkan X ke dalam daftar, sebelum posisi yang diberikan oleh iterator pos Ini adalah operasi waktu konstan untuk daftar, tapi tidak untuk vektor Nilai yang dikembalikan adalah iterator yang mewakili posisi item yang dimasukkan.
- iterator erase (iterator pos): menghapus objek pada posisi yang diberikan oleh iterator. Ini adalah operasi waktu konstan untuk daftar, tapi tidak untuk vektor Nilai yang dikembalikan adalah posisi elemen yang diikuti pos sebelum panggilan. Operasi ini membatalkan pos, yang sekarang sudah basi, karena item kontainer yang dilihatnya telah dihapus.
- iterator erase (mulai iterator, akhir iterator): menghapus semua item yang dimulai pada position start, sampai dengan, tetapi tidak termasuk akhir. Perhatikan bahwa seluruh daftar dapat dihapus dengan panggilan `c.erase( c.begin(), c.end() )`.

3.3.2 Contoh: Menggunakan penghapusan pada Daftar

Sebagai contoh, kami menyediakan rutinitas yang menghapus setiap item kedua dalam daftar, dimulai dengan item awal. Jadi, jika daftar tersebut berisi 6, 5, 1, 4, 2, maka setelah metode dipanggil, daftar tersebut akan berisi 5, 4. Kami melakukan ini dengan menelusuri daftar dan menggunakan menghapus metode pada setiap item kedua. Pada daftar, ini akan menjadi rutinitas waktu linier karena setiap panggilan ke menghapus membutuhkan waktu yang konstan, tetapi dalam vektor seluruh rutinitas akan memakan waktu kuadrat karena setiap panggilan erase tidak efisien, menggunakan $O(N)$ waktu. Akibatnya, kita biasanya akan menulis kode untuk daftar saja. Namun, untuk tujuan eksperimen, kami menulis templat fungsi umum yang akan bekerja dengan keduanya daftar atau sebuah vektor, dan kemudian menyediakan

```

1  template <typename container>
2  void removeEveryOtherItem(container & lst )
3  {
4      auto itr = lst.begin();           // itr is a Container::iterator
5
6      while (itr != lst.end() )
7      {
8          itr = lst.erase( itr );
9          if( itr != lst.end() )
10             ++ itr;
11     }
12 }

```

Gambar 3.5 Menggunakan iterator untuk menghapus setiap item lain dalam Daftar (baik vektor maupun daftar). Efisien untuk daftar, tapi tidak untuk vektor.

informasi waktu. Template fungsi ditunjukkan pada Gambar 3.5. Penggunaan mobil pada baris 4 adalah fitur C++11 yang memungkinkan kita menghindari tipe yang lebih panjang `container::iterator` jika kita menjalankan kode tersebut, melewati daftar<int>, dibutuhkan 0,039 detik untuk 800.000 item daftar, dan 0,073 detik untuk 1.600.000 item daftar, dan jelas merupakan rutinitas waktu linier, karena waktu berjalan meningkat dengan faktor yang sama dengan ukuran input. Ketika kita melewati vektor<int>, rutinitas ini memakan waktu hampir lima menit untuk 800.000 item vektor dan sekitar dua puluh menit untuk 1.600.000 item vektor; peningkatan empat kali lipat dalam waktu berjalan ketika input hanya meningkat dengan faktor dua konsisten dengan perilaku kuadrat.

3.3.3 const_iterators

Hasil dari itu bukan hanya nilai item yang dilihat oleh iterator, tetapi juga item itu sendiri. Perbedaan ini membuat iterator sangat kuat tetapi juga menimbulkan beberapa komplikasi. Untuk melihat manfaatnya, misalkan kita ingin mengubah semua item dalam koleksi ke nilai tertentu. Rutin berikut berfungsi untuk keduanya vektor Dan daftar dan berjalan dalam waktu linear. Ini adalah contoh yang bagus untuk menulis kode generik dan independen tipe.

```

template <typename container, typename Object> void
change( container & c, const Object & newvalue )
{
    typename container::iterator itr = c.begin();
    while( itr != c.end() )
        * itr++ = newvalue;
}

```

Untuk melihat potensi masalahnya, anggaplah Kontainer `c` diteruskan ke rutinitas menggunakan referensi `callby-constant`. Ini berarti kita berharap tidak ada perubahan yang diizinkan untuk `c`, dan kompiler akan memastikan hal ini dengan tidak mengizinkan panggilan ke salah satu `Cmutator`. Perhatikan kode berikut yang mencetak daftar bilangan bulat tetapi juga mencoba menyelipkan dalam perubahan ke

daftar:

```

void print( const list<int> & lst, ostream & out= cout )
{
    type name container::iterator itr = lst.begin( );
    while( itr != lst.end( ) )
    {
        out << *itr << endl;
        *itr = 0; // this is fishy!!!

        ++ itr;
    }
}

```

Jika kode ini legal, maka keteguhan daftar akan sama sekali tidak berarti, karena akan sangat mudah dilewati. Kode tersebut tidak legal dan tidak akan dikompilasi. Solusi yang diberikan oleh STL adalah bahwa setiap koleksi tidak hanya berisi pengulangan tipe bersarang tetapi juga `const_iterator` tipe bersarang. Perbedaan utama antara tipe bersarang pengulangan dan sebuah `const_iterator` apakah itu operator* untuk `const_iterator` mengembalikan referensi konstan, dan dengan demikian * itu untuk sebuah `const_iterator` tidak dapat muncul di sisi kiri pernyataan penugasan.

Selain itu, kompiler akan memaksa Anda untuk menggunakan `const_iterator` untuk melintasi koleksi konstan. Hal ini dilakukan dengan menyediakan dua versi mulai dan dua versi akhir, sebagai berikut:

- `iterator begin( )`
- `const_iterator begin( ) const`
- `iterator end( )`
- `const_iterator end( ) const`

Dua versi dari mulai dapat berada di kelas yang sama hanya karena kekonstanan suatu metode (yaitu, apakah itu aksesor atau mutator) dianggap sebagai bagian dari tanda tangan. Kita melihat trik ini di Bagian 1.7.2 dan kita akan melihatnya lagi di Bagian 1.4, keduanya dalam konteks kelebihan beban operator[].

Jika mulai dipanggil pada kontainer nonkonstan, versi “mutator” yang mengembalikan pengulangan dipanggil. Namun, jika mulai dipanggil pada kontainer konstan, yang dikembalikan adalah `const_iterator`, dan nilai pengembalian tidak dapat ditetapkan ke pengulangan. Jika Anda mencoba melakukannya, kesalahan kompiler akan muncul. Setelah itu adalah sebuah `const_iterator`, *itu=0 mudah dideteksi sebagai sesuatu yang ilegal.

Jika Anda menggunakan mobil untuk mendeklarasikan iterator Anda, kompiler akan menyimpulkan untuk Anda apakah pengulangan atau `const_iterator` diganti; sebagian besar, ini membebaskan programmer dari keharusan melacak tipe iterator yang benar dan merupakan salah satu tujuan penggunaan mobil. Selain itu, kelas perpustakaan seperti vektor dan daftar yang menyediakan iterator seperti yang dijelaskan di atas kompatibel dengan rentang berbasis untuk loop, seperti halnya kelas yang ditentukan pengguna.

Fitur tambahan di C++11 memungkinkan seseorang untuk menulis kode yang berfungsi bahkan jika wadah tipe tidak memiliki mulai dan akhir fungsi anggota. Fungsi bebas non-anggota mulai dan akhir didefinisikan yang memungkinkan seseorang untuk menggunakan mulai (c) di tempat mana pun `c.mulai( )` diperbolehkan. Menulis kode generik menggunakan `mulai(c)` alih-alih `c.mulai( )` memiliki keuntungan karena memungkinkan kode generik bekerja pada kontainer yang memiliki mulai/akhir sebagai anggota maupun mereka yang tidak memiliki mulai/akhir tetapi yang kemudian dapat ditambah dengan yang sesuai.

```

1  template <typename container>
2  void print( const container & c, ostream & out = cout )
3  {
4      if ( c.empty() )
5          out << "(empty)";
6      else
7      {
8          auto itr = begin ( c );          // itr is a Container::const_iterator
9
10         out << "[" << *itr++;           // print first item
11
12         while (itr != end (c))
13             out << ", " << *itr++;
14         out << "]" << endl;
15     }
16 }

```

Gambar 3.6 Mencetak wadah apa pun

fungsi non-anggota. Penambahan mulai dan akhir sebagai fungsi gratis di C++11 dimungkinkan dengan penambahan fitur bahasa mobil Dan decltype, seperti yang ditunjukkan pada kode di bawah ini.

```

template<typename container>
auto begin ( container & c ) -> decltype ( c.begin() )
{
    return c.begin();
}

template<typename container>
auto begin ( const container & c ) -> decltype ( c.begin() )
{
    return c.begin();
}

```

Dalam kode ini, tipe pengembalian mulai disimpulkan menjadi tipe `c.mulai()`.

Kode pada Gambar 3.6 menggunakan mobil untuk mendeklarasikan iterator (seperti pada Gambar 3.5) dan menggunakan fungsi non-anggota mulai dan akhir.

3.4 implementasi vektor

Pada bagian ini, kami menyediakan implementasi yang dapat digunakan vektor templat kelas. vektor akan menjadi tipe kelas satu, artinya tidak seperti array primitif di C++, vektor dapat disalin, dan memori yang digunakannya dapat diambil kembali secara otomatis (melalui destruksi tonya). Di Bagian 1.5.7, kami menjelaskan beberapa fitur penting dari array primitif C++:

- Array hanyalah variabel penunjuk ke blok memori; ukuran array sebenarnya harus diatur secara terpisah oleh programmer.
- Blok memori dapat dialokasikan via `new[]` tetapi kemudian harus dibebaskan melalui `hapus[]`.
- Blok memori tidak dapat diubah ukurannya (tetapi blok baru yang mungkin lebih besar dapat diperoleh dan diinisialisasi dengan blok lama, lalu blok lama dapat dibebaskan).

Untuk menghindari ambiguitas dengan kelas perpustakaan, kami akan memberi nama kelas kami `template Vektor` Sebelum memeriksa Vektor kode, kami menguraikan detail utamanya:

1. Vektor akan mempertahankan array primitif (melalui variabel pointer ke blok memori yang dialokasikan), kapasitas array, dan jumlah item saat ini yang disimpan di Vektor.
2. Vektor akan menerapkan Big-Five untuk menyediakan semantik salinan mendalam untuk konstruktor salinan dan operator`=`, dan akan menyediakan destruktur untuk mengambil kembali array primitif. Ini juga akan mengimplementasikan semantik pemindahan C++11.
3. Vektor akan memberikan mengubah ukuran rutin yang akan mengubah (umumnya ke angka yang lebih besar) ukuran Vektor dan sebuah menyimpan rutin yang akan mengubah (umumnya ke angka yang lebih besar) kapasitas Vektor. Kapasitas diubah dengan memperoleh blok memori baru untuk larik primitif, menyalin blok lama ke blok baru, dan mengambil kembali blok lama.
4. Vektor akan memberikan implementasi operator`[]` (sebagaimana disebutkan dalam Bagian 1.7.2, operator`[]` biasanya diimplementasikan dengan versi aksesor dan mutator).
5. Vektor akan menyediakan rutinitas dasar, seperti ukuran, kosong, jernih (yang biasanya berupa kalimat satu baris), `back`, `pop_back`, Dan dorong balik. Itu dorong balik rutin akan memanggil menyimpan jika ukuran dan kapasitasnya sama.
6. Vektor akan memberikan dukungan untuk tipe bersarang pengulangan dan `const_iterator`, dan terkait mulai dan akhir metode.

Gambar 3.7 dan Gambar 3.8 menunjukkan kelas Vektor. Seperti kelas STL, pemeriksaan kesalahannya terbatas. Nanti kita akan membahas secara singkat bagaimana pemeriksaan kesalahan dapat dilakukan.

Seperti yang ditunjukkan pada baris 118 hingga 120, Vektor menyimpan ukuran, kapasitas, dan array primitif sebagai anggota datanya. Konstruktor pada baris 7 hingga 9 memungkinkan pengguna untuk menentukan ukuran awal, yang secara default bernilai nol. Kemudian, konstruktor tersebut menginisialisasi anggota data, dengan kapasitas yang sedikit lebih besar daripada ukurannya, sehingga beberapa dorong baliks dapat dilakukan tanpa mengubah kapasitas.

Konstruktor salinan, yang ditunjukkan pada baris 11 hingga 17, membuat Vektor dan kemudian dapat digunakan oleh implementasi kasual operator`=` yang menggunakan idiom standar pertukaran dalam salinan. Idiom ini hanya berfungsi jika pertukaran dilakukan dengan memindahkan, yang dengan sendirinya memerlukan implementasi konstruktor pemindahan dan pemindahan operator`=` ditunjukkan pada baris 29 hingga 44. Sekali lagi, ini menggunakan idiom yang sangat standar. Implementasi penugasan salinan operator`=` menggunakan copy constructor dan swap, meskipun sederhana, tentu saja bukan metode yang paling efisien, terutama dalam kasus di mana keduanya Vektors memiliki ukuran yang sama. Dalam kasus khusus tersebut, yang dapat diuji, akan lebih efisien jika hanya menyalin setiap elemen satu per satu menggunakan Object's operator`=`.

```

1  #include <algorithm>
2
3  templat <typename Objek>
4  class Vektor
5  {
6      public:
7          explicit vector ( int initSize = 0 ): thesize{ initSize },
8              thecapacity{ intsize + SPARE CAPACITY }
9              { objects = new Object [ thecapacity ]; }
10
11      Vektor ( const Vektor & rhs ) : thesize{ rhs. thesize},
12          thecapacity{ rhs.thecapacity}, objects { nullptr }
13      {
14          objects = new Object[thecapacity];
15          for( int k
16              = 0; k < Ukuran; ++k )
17              object[ k ] = rhs.object[ k ];
18      }
19      Vektor & operator= ( const Vektor & rhs )
20      {
21          vektor copy = rhs;
22          std::swap( *this, copy );
23          return *this;
24      }
25
26      ~Vektor()
27      { delete [ ] objects; }
28
29      Vektor (Vektor && rhs): thesize {rhs.thesize },
30          thecapacity{ rhs.thecapacity }, object{ rhs.object }
31      {
32          rhs.objects = nullptr;
33          rhs.thesize = 0;
34          rhs.thecapacity = 0;
35      }
36
37      Vektor & operator= ( Vektor && rhs ) {
38
39          std::swap( thesize, rhs. thesize );
40          std::swap( thecapacity, rhs. thecapacity );
41          std::swap( objects, rhs.objects );
42
43          return *this;
44      }
45

```

Gambar 3.7 kelas vektor (Bagian 1 dari 2)

```

46     void resize ( int newsize )
47     {
48         if ( newsize > thecapacity )
49             reserve ( newsize * 2 );
50         thesize = newsize;
51     }
52
53     void reserve( int newcapacity )
54     {
55         if (newcapacity < thesize )
56             return;
57
58         Object *newArray = new Object [ newcapacity ];
59         for( int k = 0; k < theSize; ++k )
60             newArray[ k ] = std::move( objects [ k ] );
61
62         thecapacity = newCapacity;
63         std::swap( object , newArray );
64         delete [ ] newArray;
65     }

```

Gambar 3.7 (lanjutan)

mengubah ukuran rutin ditunjukkan pada baris 46 hingga 51. Kode tersebut hanya mengaturUkuran anggota data, setelah kemungkinan memperluas kapasitas. Memperluas kapasitas sangat mahal. Jadi jika kapasitas diperluas, itu dibuat dua kali lebih besar dari ukurannya untuk menghindari keharusan mengubah kapasitas lagi kecuali ukurannya meningkat secara dramatis (+1digunakan jika ukurannya 0). Perluasan kapasitas dilakukan dengan menyimpan rutin, ditunjukkan pada baris 53 hingga 65. Ini terdiri dari alokasi array baru pada baris 58, memindahkan konten lama pada baris 59 dan 60, dan reklamasi array lama pada baris 64. Seperti yang ditunjukkan pada baris 55 dan 56,menyimpanRutin juga dapat digunakan untuk mengecilkan array yang mendasarinya, tetapi hanya jika kapasitas baru yang ditentukan setidaknya sama besar dengan ukurannya. Jika tidak,menyimpanpermintaan diabaikan.

Dua versi darioperator[]adalah hal yang sepele (dan sebenarnya sangat mirip dengan implementasioperator[]di dalam matriks kelas di Bagian 1.7.2) dan ditampilkan di baris 67 hingga 70. Pemeriksaan kesalahan mudah ditambahkan dengan memastikan bahwaindeksberada dalam kisaran 0 keukuran()-1, inklusif, dan melemparkan pengecualian jika tidak.

Sejumlah rutinitas pendek, yaitu,empty,size,capacity,push_back, pop_back, Dan kembali,diimplementasikan pada baris 72 hingga 101. Pada baris 83 dan 90, kita melihat penggunaan postfix++ operator, yang menggunakan Ukuran untuk mengindeks array dan kemudian meningkatkan Ukuran Kita melihat idiom yang sama ketika membahas iterator:*itr++kegunaan itu untuk memutuskan item mana yang akan dilihat dan kemudian majuitu. Penempatan++penting: Di awalan++operator,*++itu kemajuan itu dan kemudian menggunakan yang baru itu untuk memutuskan item mana yang akan dilihat, dan juga, objek[++ukuran]akan meningkatkan Ukuran dan menggunakan nilai baru untuk mengindeks array (yang bukan merupakan hal yang kita inginkan).pop_kembali Dan kembali keduanya dapat memperoleh manfaat dari pemeriksaan kesalahan di mana pengecualian dilemparkan jika ukurannya 0.


```

67     Object & operator[]( int index )
68     { return object [index]; }
69     const Objek & operator[]( int index ) const
70     {return object [index]; }
71
72     bool empty() const
73     { return size () == 0; }
74     int size() const
75     { return thesize; }
76     int capacity() const
77     { return thecapacity; }
78
79     void push_back( const Object & x )
80     {
81         if ( thesize == thecapacity )
82             reserve( 2 * thecapacity+ 1 );
83         object [ thesize++ ] = x;
84     }
85
86     void push_back( Objek dan x ) {
87         if ( thesize == thecapacity )
88             reserve ( 2 * thecapacity + 1 );
89         objects [ thesize ++ ] = std::remove ( x );
90     }
91
92
93     void pop_back()
94     {
95         -- thesize ;
96     }
97
98     const Object & back ( ) const
99     {
100         return object [ theSize - 1 ];
101     }
102
103     typedef Object * iterator;
104     typedef const Object * const_iterator;
105
106     iterator begin()
107     { return &objects[ 0 ]; }
108     const_iterator begin() const
109     { return &objects[ 0 ]; }

```

Gambar 3.8 kelas vektor (Bagian 2 dari 2)

```

110    iterator and()
111        { return &object[ size() ]; }
112    const_iterator end() const
113        { return &objects[ size() ]; }
114
115    static const int SPARE_CAPACITY = 16;
116
117    Private:
118        int thesize;
119        int thecapacity;
120        Object * objects;
121    };

```

Gambar 1.8 (lanjutan)

Terakhir, pada baris 103 sampai 113 kita melihat deklarasi pengulangan Dan `const_iterator` tipe bersarang dan dua mulai dan dua akhir metode. Kode ini memanfaatkan fakta bahwa dalam C++, variabel pointer memiliki semua operator yang sama yang kita harapkan untuk pengulangan Variabel pointer dapat disalin dan dibandingkan; *operator menghasilkan objek yang ditunjuk, dan, yang paling aneh, ketika ++ Jika diterapkan pada variabel pointer, variabel pointer tersebut kemudian menunjuk ke objek yang akan disimpan berikutnya secara berurutan: Jika pointer menunjuk ke dalam array, penambahan pointer akan menempatkannya di elemen array berikutnya. Semantik untuk pointer ini berasal dari awal tahun 70-an dengan bahasa pemrograman C, yang menjadi dasar C++. Mekanisme iterator STL dirancang sebagian untuk meniru operasi pointer.

Oleh karena itu, pada baris 103 dan 104, kita melihat type pernyataan yang menyatakan pengulangan Dan `const_iterator` hanyalah nama lain untuk variabel pointer, dan mulai Dan akhir perlu mengembalikan alamat memori yang masing-masing mewakili posisi array pertama dan posisi array pertama yang tidak valid.

Korespondensi antara iterator dan pointer untuk vektortipe berarti menggunakan vektor alih-alih array C++, kemungkinan besar akan membawa sedikit overhead. Kerugiannya adalah, seperti yang tertulis, kode tersebut tidak memiliki pemeriksaan kesalahan. Jika iterator itu menabrak penanda akhir, tidak ada ++ itu juga bukan *itu akan memberikan sinyal kesalahan. Untuk memperbaiki masalah ini, diperlukan pengulangan Dan `const_iterator` menjadi tipe kelas bersarang yang sebenarnya, bukan sekadar variabel penunjuk. Penggunaan tipe kelas bersarang jauh lebih umum dan itulah yang akan kita lihat diDaftarkelas di Bagian 3.5.

3.5 implementasi daftar

Pada bagian ini, kami menyediakan implementasi yang dapat digunakan daftar templat kelas. Seperti halnya vektor kelas, kelas daftar kita akan diberi nama Daftar untuk menghindari ambiguitas dengan kelas perpustakaan.

Ingatlah bahwa Daftar Kelas akan diimplementasikan sebagai daftar tertaut ganda dan kita perlu mempertahankan pointer di kedua ujung daftar. Hal ini memungkinkan kita untuk mempertahankan biaya waktu per operasi yang konstan, selama operasi terjadi pada posisi yang diketahui. Posisi yang diketahui dapat berada di salah satu ujung atau pada posisi yang ditentukan oleh sebuah iterator.

Dalam mempertimbangkan desain, kita perlu menyediakan empat kelas:

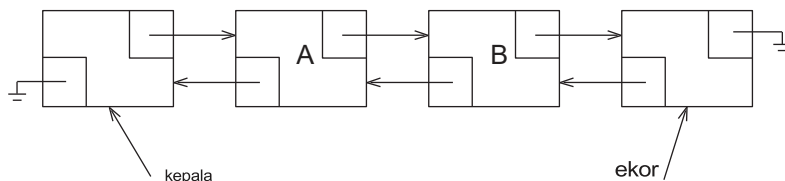
1. Itu Daftar kelas itu sendiri, yang berisi tautan ke kedua ujungnya, ukuran daftar, dan sejumlah metode.
2. Itu Simpul kelas, yang kemungkinan merupakan kelas bersarang privat. Sebuah simpul berisi data dan penunjuk ke simpul sebelumnya dan berikutnya, beserta konstruktor yang sesuai.
3. Itu `const_iterator` kelas, yang mengabstraksikan gagasan tentang suatu posisi, dan merupakan kelas bersarang publik. `const_iterator` menyimpan pointer ke node "saat ini", dan menyediakan implementasi operasi iterator dasar, semuanya dalam bentuk operator yang kelebihan beban seperti `=`, `!=`, `Dan++`.
4. Itu pengulangan kelas, yang mengabstraksikan gagasan tentang suatu posisi, dan merupakan kelas bersarang publik. `pengulangan` memiliki fungsi yang sama dengan `const_iterator`, kecuali bahwa operator `*` mengembalikan referensi ke item yang sedang dilihat, bukan referensi konstan ke item tersebut. Masalah teknis yang penting adalah bahwa pengulangan dapat digunakan dalam rutinitas apa pun yang membutuhkan `const_iterator`, tetapi tidak sebaliknya. Dengan kata lain, `pengulangan` IS-A `const_iterator`.

Karena kelas-kelas iterator menyimpan penunjuk ke "simpul saat ini", dan penanda akhir merupakan posisi yang valid, masuk akal untuk membuat simpul tambahan di akhir daftar untuk merepresentasikan penanda akhir. Selanjutnya, kita dapat membuat simpul tambahan di awal daftar, yang secara logis merepresentasikan penanda awal. Simpul tambahan ini terkadang dikenal sebagai kelenjar sentinel; khususnya, simpul di bagian depan terkadang dikenal sebagai simpul tajuk, dan simpul di ujungnya kadang-kadang dikenal sebagai **tail node**.

Keuntungan menggunakan simpul tambahan ini adalah menyederhanakan pengkodean secara signifikan dengan menghilangkan sejumlah kasus khusus. Misalnya, jika kita tidak menggunakan simpul header, maka menghapus simpul pertama menjadi kasus khusus, karena kita harus mengatur ulang tautan daftar ke simpul pertama selama penghapusan dan karena algoritma penghapusan secara umum perlu mengakses simpul tersebut sebelum simpul dihapus (dan tanpa simpul header, simpul pertama tidak memiliki simpul sebelumnya). Gambar 3.9 menunjukkan daftar tertaut ganda dengan simpul header dan ekor. Gambar 3.10 menunjukkan daftar kosong.

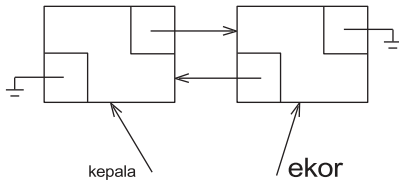
Gambar 3.11 dan Gambar 3.12 menunjukkan garis besar dan implementasi parsial dari Daftar kelas.

Kita dapat melihat pada baris ke-5 awal deklarasi `private nestedSimpulkelas`. Daripada menggunakan kelas kata kunci, kita menggunakan struktur. Dalam C++, struktur adalah peninggalan



dari bahasa pemrograman C. Struktur dalam C++ pada dasarnya adalah kelas di mana anggotanya default ke publik. Ingat bahwa dalam kelas, anggota default ke pribadi. Jelas struktur.

Gambar 3.9 Daftar tertaut ganda dengan simpul header dan ekor



Gambar 3.10 Daftar tertaut ganda kosong dengan simpul header dan ekor

Kata kunci tidak diperlukan, tetapi Anda akan sering melihatnya dan umumnya digunakan oleh programmer untuk menandakan tipe yang sebagian besar berisi data yang diakses secara langsung, alih-alih melalui metode. Dalam kasus kami, menjadikan anggota publik diSimpul kelas tidak akan menjadi masalah, karena Simpul kelas itu sendiri bersifat pribadi dan tidak dapat diakses di luar Daftar kelas.

Pada baris 9 kita melihat awal deklarasi publik bersarang `const_iterator` kelas, dan pada baris 12 kita melihat awal deklarasi publik bersarang pengulangan kelas. Sintaks yang tidak biasa adalah warisan, yang merupakan konstruksi kuat yang tidak digunakan dalam buku ini. Sintaks pewarisan menyatakan bahwa pengulangan memiliki fungsi yang sama persis dengan `const_iterator`, dengan kemungkinan beberapa tambahan, dan itu pengulangan kompatibel dengan tipe `const_iterator` dan dapat digunakan dimana saja `const_iterator` di perlukan. Kita akan membahas detailnya nanti saat kita melihat implementasinya yang sebenarnya.

Baris 80 hingga 82 berisi anggota data untuk Daftar, yaitu, penunjuk ke node header dan tail. Kami juga melacak ukuran anggota data sehingga ukuran metode dapat diimplementasikan dalam waktu yang konstan.

Sisa dari Daftar Kelas terdiri dari konstruktor, Big-Five, dan sejumlah metode. Banyak metode yang berupa kalimat satu baris. Mulai dan akhirnya mengembalikan iterator yang sesuai; panggilan pada baris 30 adalah tipikal implementasi, di mana kita mengembalikan iterator yang dibangun (dengan demikian pengulangan Dan `const_iterator` Setiap kelas memiliki konstruktor yang mengambil pointer ke simpul sebagai parameternya).

Itu clear metode pada baris 43 sampai 47 bekerja dengan menghapus item berulang kali sampai Daftar kosong. Menggunakan strategi ini memungkinkan jernih untuk menghindari tangannya kotor dalam mengambil kembali node karena pengambilan kembali node sekarang disalurkan ke `pop_front` Metode pada baris 48 hingga 67 semuanya bekerja dengan cara mendapatkan dan menggunakan iterator yang tepat secara cerdas. Ingat bahwa menyisipkan metode menyisipkan sebelum suatu posisi, jadi dorong balok sisipan sebelum penanda akhir, sesuai kebutuhan. Di `pop_back`, perhatikan bahwa `hapus(-akhir())` membuat iterator sementara yang sesuai dengan endmarker, menarik kembali iterator sementara, dan menggunakan iterator tersebut untuk menghapus. Perilaku serupa terjadi dikembali. Perhatikan juga bahwa dalam kasus `pop_front` Dan `pop_back` operasi, kita sekali lagi menghindari penanganan reklamasi node.

Gambar 1.13 menunjukkan kelas Node, yang terdiri dari item yang disimpan, pointer ke item sebelumnya dan berikutnya. Simpul, dan konstruktor. Semua anggota data bersifat publik.

Gambar 1.14 menunjukkan kelas `const_iterator`, dan Gambar 1.15 menunjukkan kelas iterator. Seperti yang telah disebutkan sebelumnya, sintaksis pada baris ke-39 pada Gambar 1.15 menunjukkan fitur lanjutan yang dikenal sebagai warisan dan berarti bahwa pengulangan `IS-A const_iterator` Ketika pengulangan kelas ditulis dengan cara ini, ia mewarisi semua data dan metode dari `const_iterator`. Kemudian, ia dapat menambahkan data baru, menambahkan metode baru, dan mengganti (yaitu, mendefinisikan ulang) metode yang sudah ada. Dalam skenario paling umum, terdapat beban sintaksis yang signifikan (seringkali mengakibatkan kata kunci maya (muncul dalam kode)).

```

1  template<typename Object>
2  class list
3  {
4      private:
5          struct Node
6              { /* see figure 1.13 */ };
7
8      public:
9          class const_iterator
10             { /* see figure 3.14 */ };
11
12         class iterator : public const_iterator
13             { /* see figure 1.15 */ };
14
15     public:
16         list()
17             { /* see figure 1.16 */ }
18         List( const List & rhs )
19             { /* see figure 1.16 */ }
20         ~List()
21             { /* see figure 1.16 */ }
22         list & operator= ( const list & rhs )
23             { /* see figure 1.16 */ }
24         List( List && rhs )
25             { /* see figure 1.16 */ } list
26         & operator= ( list && rhs )
27             { /* see figure 1.16 */ }
28
29         iterator mulai()
30             { return { head->next }; }
31         const_iterator begin() const
32             { return { head-next }; }
33         iterator end()
34             { return { tail }; }
35         const_iterator end() const
36             { return { tail }; }
37
38         int size() const
39             { return thesize; }
40         bool empty() const
41             { return size() == 0; }
42
43         void clear()
44         {
45             while( !empty() )
46                 pop_front();
47     }

```

Gambar 3.11 Kelas daftar (Bagian 1 dari 2)

```

48     Object & front()
49         { return *begin(); } const
50     Object & front() const
51         { return *begin(); }
52     Object & back()
53         { return *--end(); }
54     const Object & back() const
55         { return *--end(); }
56     void push_front( const Object& x )
57         { insert( begin(), x ); }
58     void push_front( Object dan x )
59         { insert( begin(), std::move( x ) ); }
60     void push_back( const Object & x )
61         { insert( end(), x ); }
62     void push_back( Object && x )
63         { insert( end(), std::move( x ) ); }
64     void pop_front()
65         { erase( begin() ); }
66     void pop_back()
67         { erase( --end() ); }
68
69     iterator insert(iterator itr, const Object & x)
70         { /* see figure 1.18 */ }
71     iterator insert(iterator itr, Objek && x)
72         { /* see figure 1.18 */ }
73
74     iterator erase (iterator itr)
75         { /* see figure 1.20 */ }
76     iterator erase(iterator from, iterator to)
77         { /* see figure 1.20 */ }
78
79     private:
80         int thesize
81         node *head;
82         node *tail;
83
84     void init()
85         { /* see figure 1.16 */ }
86 };

```

Gambar 3.12 Kelas daftar (Bagian 2 dari 2)

```

1      struct Node
2      {
3          Object    data;
4          Node     * prev;
5          Node     * next;
6
7          Node( const Object & d = Object{ }, Node * p = nullptr,
8                                     Node * n = nullptr )
9              : data{ d }, prev{ p }, next{ n }{ }
10
11         Node( Object && d, Node * p = nullptr, Node * n = nullptr )
12             : data{ std::move( d ) }, prev{ p }, next{ n }{ }
13     };

```

Gambar 3.13 Kelas Node Bersarang untuk kelas Daftar

Namun, dalam kasus kami, kami dapat menghindari banyak kerumitan sintaksis karena kami tidak menambahkan data baru, juga tidak bermaksud mengubah perilaku metode yang sudah ada. Namun, kami menambahkan beberapa metode baru dipengulangan kelas (dengan tanda tangan yang sangat mirip dengan metode yang ada di `const_iterator` kelas). Akibatnya, kita dapat menghindari penggunaan `virtual`. Meskipun begitu, ada beberapa trik sintaksis `dconst_iterator`.

Pada baris 28 dan 29, `const_iterator` menyimpan sebagai anggota data tunggalnya sebuah pointer ke node “saat ini”. Biasanya, ini akan bersifat pribadi, tetapi jika bersifat pribadi, maka pengulangan tidak akan memiliki akses ke sana. Menandai anggota `const_iterator` sebagai terlindungi memungkinkan kelas yang mewarisi dari `const_iterator` untuk memiliki akses terhadap anggota tersebut, tetapi tidak mengizinkan kelas lain untuk memiliki akses.

Pada baris 34 dan 35 kita melihat konstruktor untuk `const_iterator` yang digunakan dalam `Daftar` implementasi kelas mulai Dan akhir. Kami tidak ingin semua kelas melihat konstruktor ini (iterator tidak seharusnya dibuat secara kasat mata dari variabel pointer), jadi tidak bisa bersifat publik, tetapi kami juga ingin pengulangan kelas untuk dapat melihatnya, jadi secara logis konstruktor ini dibuat terlindungi. Namun, ini tidak memberikan `Daftar` akses ke konstruktor. Solusinya adalah deklarasi teman pada baris 37, yang memberikan `Daftar` akses kelas ke `const_iterator` anggota nonpublik.

Metode publik dalam `const_iterator` semuanya menggunakan operator yang kelebihan beban. `operator==`, `operator!=`, Dan `operator*` sangat mudah. Pada baris 10 sampai 21 kita melihat penerapan `operator++`. Ingat bahwa versi awalan dan akhiran dari `operator++` sangat berbeda dalam semantik (dan preseden), jadi kita perlu menulis rutin terpisah untuk setiap bentuk. Rutin-rutin tersebut memiliki nama yang sama, sehingga harus memiliki tanda tangan yang berbeda agar dapat dibedakan. C++ mengharuskan kita untuk memberikan tanda tangan yang berbeda dengan menentukan daftar parameter kosong untuk bentuk awalan dan satu (anonim) ke dalam parameter untuk bentuk postfix. Kemudian `++` itu memanggil parameter nol `operator++`; Dan `itr++` memanggil satu parameter `operator++`. Itu ke dalam Parameter ini tidak pernah digunakan; hanya ada untuk memberikan tanda tangan yang berbeda. Implementasinya menunjukkan bahwa, dalam banyak kasus di mana terdapat pilihan antara menggunakan awalan atau akhiran, `operator++`, bentuk awalan akan lebih cepat daripada bentuk akhiran.

Di dalam pengulangan kelas, konstruktor yang dilindungi pada baris 64 menggunakan daftar inisialisasi untuk menginisialisasi simpul saat ini yang diwarisi. Kita tidak perlu mengimplementasikan ulang `operator==`

```

1  class const_iterator
2  {
3      public:
4          const_iterator() : current{ nullptr }
5              { }
6
7          const Object & operator* () const
8              { return retrieve(); }
9
10         const_iterator & operator++ () {
11
12             current = current->next;
13             return *this;
14         }
15
16         const_iterator operator++ (int)
17         {
18             const_iterator old = *this;
19             ++ ( *this );
20             return old;
21         }
22
23         bool operator== ( const const_iterator & rhs ) const
24             { return current == rhs.current; }
25         bool operator!= ( const const_iterator & rhs ) const
26             { return !( *this == rhs ); }
27
28     protected:
29         Node *current;
30
31         Object & retrieve() const
32             { return current->data; }
33
34         const_iterator( Node *p ) : current{ p }
35             { }
36
37         friend class List<Object>;
38     };

```

Gambar 3.14 Konstanta bersarang_kelas iterator untuk kelas List

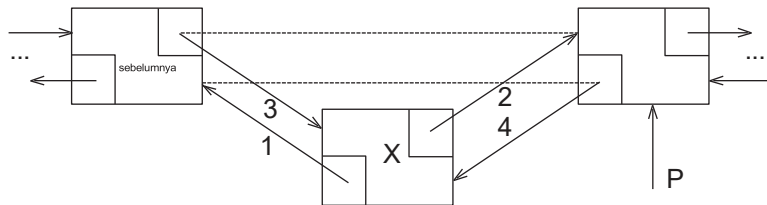
Dan operator!= karena itu diwariskan tanpa perubahan. Kami menyediakan sepasang baru operator ++ implementasi (karena tipe pengembalian yang berubah) yang menyembunyikan dokumen asli di const_iterator dan kami menyediakan pasangan aksesor/mutator untuk operator*. Aksesor operator*, ditunjukkan pada baris 47 dan 48, hanya menggunakan implementasi yang sama seperti di const_iterator. Aksesor diimplementasikan secara eksplisit dipengulang karena jika tidak, implementasi asli akan disembunyikan oleh versi mutator yang baru ditambahkan.


```

1      list( )
2          { init(); }
3
4      ~list()
5      {
6          clear( );
7          delete head;
8          delete tail;
9      }
10
11     list( const list & rhs ) {
12
13         init();
14         for ( auto & x : rhs )
15             push_back( x );
16     }
17
18     list & operator= ( const list & rhs ) {
19
20         list copy = rhs;
21         std::swap( *this, copy );
22         return *this;
23     }
24
25
26     List( List && rhs )
27         : thesize{ rhs.thesize}, head{ rhs.head}, tail{ rhs.tail}
28     {
29         rhs.thesize = 0;
30         rhs.head = nullptr;
31         rhs.tail = nullptr;
32     }
33
34     list & operator= ( list && rhs )
35     {
36         std::swap(thesize, rhs.thesize);
37         std::swap( head, rhs.head);
38         std::swap( tail, rhs.tail);
39
40         return *this;
41     }
42
43     void init()
44     {
45         thesize = 0;
46         head = new Node;
47         tail = new Node;
48         head->next = tail;
49         tail->prev = head;
50     }

```

Gambar 3.16 Konstruktor, Big-Five, dan rutinitas inisialisasi pribadi untuk kelas Daftar



Gambar 3.17 Penyisipan dalam daftar tertaut ganda dengan mendapatkan simpul baru dan kemudian mengubahnya

petunjuk dalam urutan yang ditunjukkan

Langkah 3 dan 4 dapat digabungkan, menghasilkan hanya dua baris:

```
Node *newNode = newNode{ x, p-           // step 1 dan 2 //
>prev, p }; p->prev = p->prev-         step 3 dan 4
>next = newNode;
```

Namun kedua baris ini juga dapat digabungkan, menghasilkan:

```
p->prev = p->prev->next = new Node{ x, p->prev, p };
```

Hal ini membuat menyisipkan rutin pada Gambar 3.18 pendek.

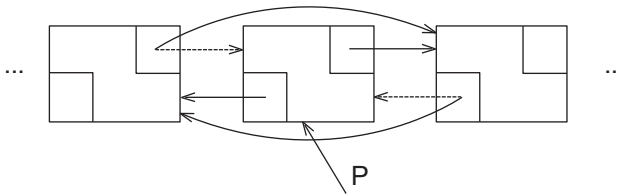
Gambar 3.19 menunjukkan logika penghapusan node. Jika P menunjuk ke node yang dihapus, hanya dua petunjuk yang berubah sebelum node dapat diambil kembali:

```
p->prev->next = p->next; p-
>next->prev = p->prev;
delete p;
```

Gambar 3.20 menunjukkan sepasang menghapus rutinitas. Versi pertama dari menghapus berisi tiga baris kode yang ditunjukkan di atas dan kode untuk mengembalikan pengulangan mewakili item setelah

```
1      // insert x before itr.
2      iterator insert(iterator itr, const Objek & x) {
3
4          Node *p = itr.current;
5          thesize++;
6          return { p->prev = p->prev->next = newNode{ x, p->prev, p } };%
7      }
8
9      // insert x before itr.
10     iterator insert(iterator itr, Object && x) {
11
12         Node *p = itr.current;
13         thesize++;
14         return { p->prev = p->prev->next
15                 = new Node{ std::move( x ), p->prev, p } };
16     }
```

Gambar 1.18 rutinitas penyisipan untuk kelas Daftar



Gambar 3.19 Menghapus node yang ditentukan oleh p dari daftar tertaut ganda

```

1      // erase item at itr.
2      iterator erase( iterator itr ) {
3
4          Node *p = itr.current;
5          iterator retVal{ p->next };
6          p->prev->next = p->next;
7          p->next p->prev=p->prev ;
8          Delete p;
9          theSize--;
10
11         return retVal;
12     }
13
14     iterator erase(iterator from, iterator to)
15     {
16         for (iterator itr = from; itr != to; )
17             itr = erase( itr );
18
19         return to;
20     }

```

Gambar 3.20 routine penghapus untuk kelas Daftar

elemen yang terhapus. Seperti menyisipkan, menghapus harus memperbarui UkuranVersi kedua dari menghapusnya menggunakan pengulangan untuk memanggil versi pertama menghapus. Perlu dicatat bahwa kita tidak bisa begitu saja menggunakan `itr++` di dalam untuk loop pada baris 16 dan abaikan nilai pengembalian menghapus pada baris 17. Nilai dari itu sudah basi segera setelah panggilan kemenghapus, itulah sebabnya menghapus mengembalikan pengulangan.

Saat memeriksa kode, kita dapat melihat banyak kesalahan yang mungkin terjadi dan tidak ada pemeriksaan yang dilakukan. Misalnya, iterator yang diteruskan kemenghapus Dan menyisipkan bisa tidak diinisialisasi atau untuk daftar yang salah! Iterator dapat memiliki `++` atau `*` diterapkan padanya saat mereka sudah berada di penanda akhir atau belum diinisialisasi.

Iterator yang belum diinisialisasi akan memiliki saat ini menunjuk ke `nullptr`, sehingga kondisinya mudah diuji. Penanda akhir Berikutnya penunjuk menunjuk ke `nullptr`, jadi pengujian untuk `++` atau `*` pada kondisi endmarker juga mudah. Namun, untuk menentukan apakah sebuah iterator diteruskan ke menghapus atau menyisipkan adalah sebuah iterator untuk daftar yang benar, iterator harus menyimpan anggota data tambahan yang mewakili sebuah pointer ke Daftar dari mana ia dibangun.

```

1    protected:
2        const List<Object> *thelist;
3        Node*current;
4
5        const_iterator( const list<Object> & lst, Node *p )
6            : theList{ &lst }, current{ p }
7        {
8        }
9
10       void assertIsValid() const
11       {
12           if (thelist == nullptr || current == nullptr || current == theList->head)
13               throw IteratorOutOfBoundsException{ };
14       }

```

Gambar 1.21 Bagian yang dilindungi dari const yang direvisi_iterator yang menggabungkan kemampuan untuk melakukan pemeriksaan kesalahan tambahan

Kita akan membuat sketsa ide dasar dan meninggalkan detailnya sebagai latihan. const_iterator kelas, kita menambahkan pointer ke Daftar dan memodifikasi konstruktor yang dilindungi untuk mengambil daftar sebagai parameter. Kita juga bisa menambahkan metode yang melempar pengecualian jika pernyataan tertentu tidak terpenuhi. Bagian protected yang direvisi terlihat seperti kode pada Gambar 3.21. Kemudian semua panggilan kepengulangan dan const_iterator konstruktor yang sebelumnya mengambil satu parameter sekarang mengambil dua, seperti pada mulai metode untuk daftar:

```

const_iterator begin() const {

    const_iterator itr{ *this, head };
    return ++itr;
}

```

Kemudian, sisipan dapat direvisi agar terlihat seperti kode pada Gambar 3.22. Detail modifikasi ini akan kami jadikan sebagai latihan.

```

1        // Insert x before itr.
2        iterator insert(iterator itr, const Object & x) {
3
4            itr.assertIsValid();
5            if (itr.thelist != this)
6                throw IteratorMismatchException{ };
7
8            Node *p = itr.current;
9            theSize++;
10           return { *this, p->prev = p->prev->next = new Node{ x, p->prev, p } };
11       }

```

Gambar 1.22 Penyisipan daftar dengan pemeriksaan kesalahan tambahan

1.6 Tumpukan ADT

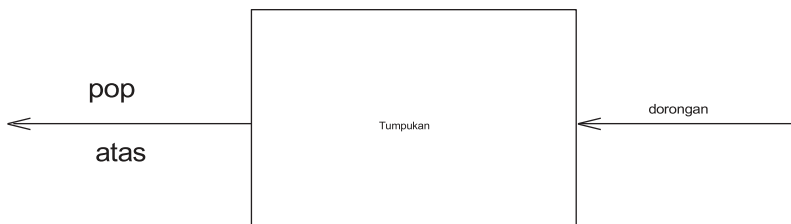
tumpukan adalah daftar dengan batasan bahwa penyisipan dan penghapusan hanya dapat dilakukan pada satu posisi, yaitu akhir daftar, yang disebutatas.

3.6.1 Model Tumpukan

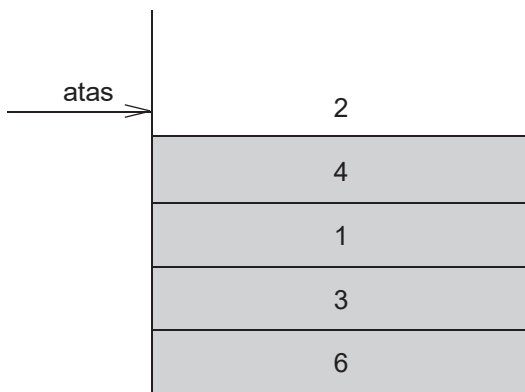
Operasi dasar pada tumpukan adalah dorongan, yang setara dengan sisipan, danpop, yang menghapus elemen yang baru saja dimasukkan. Elemen yang baru saja dimasukkan dapat diperiksa sebelum melakukan pop dengan menggunakan atas rutin. Sebuah pop atau atas pada tumpukan kosong umumnya dianggap sebagai kesalahan dalam ADT tumpukan. Di sisi lain, kehabisan ruang saat melakukan dorongan adalah batasan implementasi tetapi bukan kesalahan ADT.

Tumpukan terkadang dikenal sebagai LIFO((masuk terakhir, keluar pertama). Model yang digambarkan pada Gambar 3.23 hanya menunjukkan bahwa push adalah operasi input, sedangkan pop dan top adalah output. Operasi umum untuk mengosongkan tumpukan dan menguji kekosongan adalah bagian dari repertoar, tetapi pada dasarnya semua yang dapat Anda lakukan pada tumpukan adalah dorongan dan pop.

Gambar 3.24 menunjukkan tumpukan abstrak setelah beberapa operasi. Model umumnya adalah terdapat beberapa elemen yang berada di puncak tumpukan, dan elemen tersebut merupakan satu-satunya elemen yang terlihat.



Gambar 3.23 Model tumpukan: Input ke tumpukan dilakukan dengan push;outputnya adalah pop dan top



Gambar 3.24 Stack model : Hanya elemen teratas yang dapat diakses

3.6.2 Implementasi Tumpukan

Karena tumpukan adalah daftar, implementasi daftar apa pun bisa digunakan. Jelas daftar Dan vektor Mendukung operasi tumpukan; 99% dari waktu, operasi ini merupakan pilihan yang paling masuk akal. Terkadang, merancang implementasi khusus bisa lebih cepat. Karena operasi tumpukan adalah operasi waktu konstan, hal ini kecil kemungkinannya menghasilkan peningkatan yang nyata kecuali dalam keadaan yang sangat unik.

Untuk saat-saat khusus ini, kami akan memberikan dua implementasi tumpukan yang populer. Satu menggunakan struktur tertaut, dan yang lainnya menggunakan array, dan keduanya menyederhanakan logika dalam vektor Dan daftar, jadi kami tidak menyediakan kode.

Implementasi Daftar Tertaut dari Tumpukan

Implementasi pertama tumpukan menggunakan daftar tertaut tunggal. Kami melakukan dorongan dengan menyisipkan di bagian depan daftar. Kami melakukan pop dengan menghapus elemen di depan daftar. Atas operasi hanya memeriksa elemen di depan daftar, mengembalikan nilainya. Terkadang pop dan atas operasi digabungkan menjadi satu.

Implementasi Array dari Tumpukan

Implementasi alternatif menghindari tautan dan mungkin merupakan solusi yang lebih populer. Implementasi ini menggunakan kembali, dorong balik, Dan pop kembali implementasi dari vektor, jadi implementasinya mudah. Terkait dengan setiap tumpukan adalah Array dan atasTumpukan, yang bernilai -1 untuk tumpukan kosong (ini adalah cara tumpukan kosong diinisialisasi). Untuk mendorong beberapa elemen X ke tumpukan, kami menambah atas Tumpukan dan kemudian mengatur theArray[topofstack]=X Untuk pop, kita atur kembalinya nilai untuk the Array[atasTumpukan] dan kemudian mengurangi atas Tumpukan.

Perhatikan bahwa operasi ini dilakukan tidak hanya dalam waktu konstan, tetapi juga dalam waktu konstan yang sangat cepat. Pada beberapa mesin, dorongan es dan pop s (bilangan bulat) dapat ditulis dalam satu instruksi mesin, yang beroperasi pada register dengan pengalamatan auto-increment dan auto-decrement. Fakta bahwa sebagian besar mesin modern memiliki operasi tumpukan sebagai bagian dari set instruksi memperkuat gagasan bahwa tumpukan mungkin merupakan struktur data paling fundamental dalam ilmu komputer, setelah array.

3.6.3 Aplikasi

Tidak mengherankan jika kita membatasi operasi yang diizinkan dalam sebuah daftar, operasi-operasi tersebut dapat dilakukan dengan sangat cepat. Namun, kejutan besarnya adalah bahwa sejumlah kecil operasi yang tersisa begitu kuat dan penting. Kami akan memberikan tiga dari sekian banyak aplikasi tumpukan. Aplikasi ketiga memberikan wawasan mendalam tentang bagaimana program diorganisasikan.

Simbol Penyeimbang

Kompiler memeriksa program Anda untuk mencari kesalahan sintaksis, tetapi sering kali kekurangan satu simbol (seperti tanda kurung kurawal atau awal komentar yang hilang) dapat menyebabkan kompilator mengeluarkan ratusan baris diagnostik tanpa mengidentifikasi kesalahan sebenarnya.

Alat yang berguna dalam situasi ini adalah program yang memeriksa apakah semuanya seimbang. Jadi, setiap kurung siku, kurung siku, dan kurung kurawal kanan harus sesuai dengan pasangannya di kiri.

Urutan `[]` adalah sah, tapi `[ ( ) ]` salah. Tentu saja, menulis program yang sangat besar untuk ini tidak sepadan, tetapi ternyata mudah untuk memeriksanya. Untuk menyederhanakannya, kita hanya akan memeriksa keseimbangan tanda kurung, kurung siku, dan kurung kurawal, lalu mengabaikan karakter lain yang muncul.

Algoritma sederhana menggunakan tumpukan dan adalah sebagai berikut:

Buat tumpukan kosong. Baca karakter hingga akhir berkas. Jika karakter tersebut merupakan simbol pembuka, masukkan ke dalam tumpukan. Jika merupakan simbol penutup dan tumpukan kosong, laporkan kesalahan. Jika tidak, keluarkan tumpukan. Jika simbol yang dikeluarkan bukan simbol pembuka yang sesuai, laporkan kesalahan. Di akhir berkas, jika tumpukan tidak kosong, laporkan kesalahan.

Anda harus bisa meyakinkan diri sendiri bahwa algoritma ini berfungsi. Algoritma ini jelas linear dan hanya melakukan satu kali lintasan input. Dengan demikian, algoritma ini daring dan cukup cepat. Pekerjaan tambahan dapat dilakukan untuk mencoba memutuskan apa yang harus dilakukan ketika kesalahan dilaporkan—misalnya, mengidentifikasi kemungkinan penyebabnya.

Ekspresi Postfix

Misalkan kita memiliki kalkulator saku dan ingin menghitung biaya belanja. Untuk melakukannya, kita tambahkan sejumlah angka dan kalikan hasilnya dengan 1,06; ini menghitung harga beli beberapa barang dengan pajak penjualan lokal. Jika harga barangnya adalah 4,99, 5,99, dan 6,99, maka cara alami untuk memasukkannya adalah dengan urutan:

$$4,99 + 5,99 + 6,99 * 1,06 =$$

Tergantung kalkulatornya, ini menghasilkan jawaban yang diinginkan, 19,05, atau jawaban ilmiah, 18,39. Kebanyakan kalkulator empat fungsi sederhana akan memberikan jawaban pertama, tetapi banyak kalkulator tingkat lanjut yang mengetahui bahwa perkalian memiliki prioritas lebih tinggi daripada penjumlahan.

Di sisi lain, ada beberapa barang yang dikenakan pajak dan ada yang tidak, jadi jika hanya barang pertama dan terakhir yang benar-benar dikenakan pajak, maka urutannya

$$4,99 * 1,06 + 5,99 + 6,99 * 1,06 =$$

akan memberikan jawaban yang benar (18,69) pada kalkulator ilmiah dan jawaban yang salah (19,37) pada kalkulator sederhana. Kalkulator ilmiah umumnya dilengkapi tanda kurung, sehingga kita selalu bisa mendapatkan jawaban yang benar dengan menggunakan tanda kurung, tetapi dengan kalkulator sederhana kita perlu mengingat hasil antara.

Urutan evaluasi tipikal untuk contoh ini mungkin adalah mengalikan 4,99 dan 1,06, menyimpan jawaban ini sebagai *A1*. Kemudian kita tambahkan 5,99 dan *A1*, menyimpan hasilnya di *A1*. Kita kalikan 6,99 dan 1,06, simpan jawabannya di *A2*, dan diakhiri dengan menambahkan *A1* dan *A2*, meninggalkan jawaban akhir di *A1*. Kita dapat menuliskan urutan operasi ini sebagai berikut:

$$4,99 \ 1,06 * 5,99 + 6,99 \ 1,06 * +$$

Notasi ini dikenal sebagai postfix, atau notasi Polandia terbalik, dan dievaluasi persis seperti yang telah kami jelaskan di atas. Cara termudah untuk melakukan ini adalah dengan menggunakan tumpukan. Ketika sebuah angka terlihat, angka tersebut akan dimasukkan ke dalam tumpukan; ketika sebuah operator terlihat, operator tersebut akan diterapkan ke

dua angka (simbol) yang dikeluarkan dari tumpukan, dan hasilnya dimasukkan ke dalam tumpukan. Misalnya, ekspresi postfix

6 5 2 3 + 8 * + 3 + *

dievaluasi sebagai berikut:

Empat simbol pertama ditempatkan pada tumpukan. Tumpukan yang dihasilkan adalah

atasTumpukan →	3
	2
	5
	6

Berikutnya, tanda '+' dibaca, sehingga 3 dan 2 dikeluarkan dari tumpukan, dan jumlahnya, 5, dimasukkan.

atasTumpukan →	5
	5
	6

Berikutnya, angka 8 didorong.

atasTumpukan →	8
	5
	5
	6

Sekarang sebuah '*' terlihat, jadi 8 dan 5 muncul, dan $5 * 8 = 40$ didorong.

atasTumpukan →	40
	5
	6

Berikutnya, tanda '+' terlihat, jadi 40 dan 5 muncul, dan $5 + 40 = 45$ didorong.

atasTumpukan→	45 6
---------------	---------

Sekarang, 3 didorong.

atasTumpukan→	3 45 6
---------------	--------------

Berikutnya, '+' memunculkan 3 dan 45 dan mendorong $45 + 3 = 48$.

atasTumpukan→	48 6
---------------	---------

Akhirnya, sebuah '*' terlihat dan 48 dan 6 muncul; hasilnya, $6 * 48 = 288$, didorong.

atasTumpukan→	288
---------------	-----

Waktu untuk mengevaluasi ekspresi postfix adalah $O(N)$, karena pemrosesan setiap elemen dalam input terdiri dari operasi tumpukan dan karenanya membutuhkan waktu yang konstan. Algoritme untuk melakukannya sangat sederhana. Perhatikan bahwa ketika suatu ekspresi diberikan dalam notasi postfix, tidak perlu mengetahui aturan prioritas apa pun; ini merupakan keuntungan yang jelas.

Konversi Infix ke Postfix

Tumpukan tidak hanya dapat digunakan untuk mengevaluasi ekspresi postfix, tetapi kita juga dapat menggunakan tumpukan untuk mengonversi ekspresi dalam bentuk standar (atau dikenal sebagai infiks) ke dalam postfix. Kita akan fokus pada versi kecil dari masalah umum dengan hanya mengizinkan operator $+$, $*$, $($, dan $)$, dan tetap berpegang pada aturan prioritas yang biasa. Selanjutnya, kita akan berasumsi bahwa ekspresi tersebut legal. Misalkan kita ingin mengonversi ekspresi infiks

$$a + b * c + (d * e + f) * g$$

ke postfix. Jawaban yang benar adalah $bc * + de * f + g * +$.

Ketika sebuah operan dibaca, operan tersebut langsung ditempatkan ke keluaran. Operator tidak langsung dikeluarkan, sehingga harus disimpan di suatu tempat. Hal yang benar untuk dilakukan adalah menempatkan operator yang telah dilihat, tetapi tidak ditempatkan pada keluaran, ke dalam tumpukan. Kita juga akan menumpuk tanda kurung kiri ketika ditemukan. Kita mulai dengan tumpukan yang awalnya kosong.

Jika kita melihat simbol lain ($+$, $*$, $($), lalu kita keluarkan entri dari tumpukan hingga kita menemukan entri dengan prioritas lebih rendah. Satu pengecualian adalah kita tidak pernah menghapus entri (dari tumpukan kecuali saat memproses). Untuk tujuan operasi ini, $+$ memiliki prioritas terendah dan $($ tertinggi. Setelah popping selesai, kami mendorong operator ke tumpukan.

Akhirnya, jika kita membaca akhir masukan, kita memunculkan tumpukan hingga kosong, dan menulis simbol pada keluaran.

Ide di balik algoritma ini adalah ketika sebuah operator terlihat, operator tersebut akan ditempatkan di tumpukan. Tumpukan tersebut merepresentasikan operator yang tertunda. Namun, beberapa operator di tumpukan yang memiliki prioritas tinggi kini diketahui telah selesai dan harus dihapus, karena operator tersebut tidak lagi tertunda. Jadi, sebelum menempatkan operator di tumpukan, operator yang ada di tumpukan, dan yang harus diselesaikan sebelum operator saat ini, akan dihapus. Hal ini diilustrasikan dalam tabel berikut:

Ekspresi	Tumpuk Saat Ketiga Operator Diproses	Tindakan
$a*b-c+d$	-	- selesai; + didorong Tidak ada
$a/b+c*d$	+	yang selesai; * didorong
$ab*c/d$	- *	* selesai; / didorong
$ab*c+d$	- *	* dan - selesai; + didorong

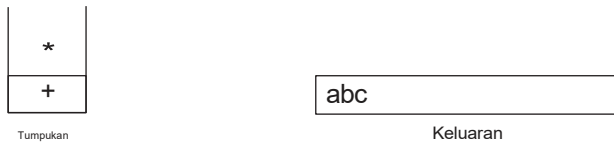
Tanda kurung hanya menambah kerumitan. Kita dapat menganggap tanda kurung kiri sebagai operator berprioritas tinggi ketika berupa simbol input (agar operator yang tertunda tetap tertunda) dan operator berprioritas rendah ketika berada di tumpukan (agar tidak terhapus secara tidak sengaja oleh operator). Tanda kurung kanan diperlakukan sebagai kasus khusus.

Untuk melihat kinerja algoritma ini, kita akan mengonversi ekspresi infiks panjang di atas ke dalam bentuk postfixnya. Pertama, simbol A dibaca, sehingga diteruskan ke output.

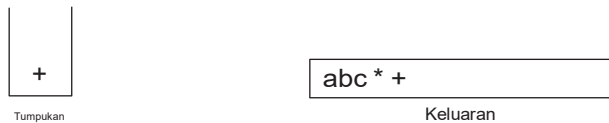
Kemudian $+$ dibaca dan dimasukkan ke dalam tumpukan. Selanjutnya B dibaca dan diteruskan ke output. Keadaan pada titik ini adalah sebagai berikut:



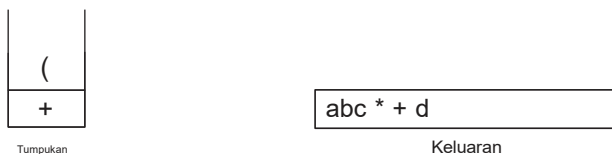
Selanjutnya, sebuah $*$ dibaca. Entri teratas pada tumpukan operator memiliki prioritas lebih rendah daripada $*$, jadi tidak ada yang dikeluarkan dan $*$ diletakkan di tumpukan. Selanjutnya, C dibaca dan dikeluarkan. Sejauh ini, kita telah



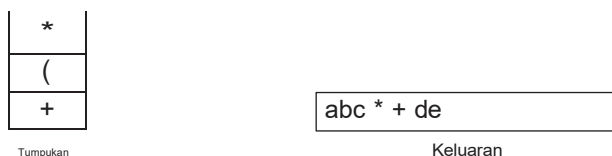
Simbol berikutnya adalah $+$. Memeriksa tumpukan, kami menemukan bahwa kami akan memunculkan $*$ dan letakkan pada output; pop yang lain $+$, yang bukan dari lebih rendah tetapi prioritas yang sama, di tumpukan; dan kemudian dorong $+$.



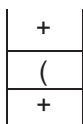
Simbol selanjutnya yang dibaca adalah $($ (Karena memiliki prioritas tertinggi, ini ditempatkan di tumpukan. Kemudian D dibaca dan dikeluarkan.



Kita lanjutkan dengan membaca $*$ (Karena tanda kurung buka tidak dihapus kecuali saat tanda kurung tutup sedang diproses, tidak ada keluaran. Selanjutnya, e dibaca dan dikeluarkan.



Simbol selanjutnya yang dibaca adalah +. Kami muncul dan mengeluarkan + lalu dorong +. Kemudian kita membaca dan mengeluarkan F.



Tumpukan

$$abc * + de * f$$

Keluaran

Sekarang kita membaca sebuah +, sehingga tumpukan dikosongkan kembali ke (Kami mengeluarkan +).

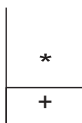


Tumpukan

$$abc * + de * f +$$

Keluaran

Kami membaca sebuah * selanjutnya; itu didorong ke tumpukan. Kemudian G dibaca dan dikeluarkan.



Tumpukan

$$abc * + de * f + g$$

Keluaran

Input sekarang kosong, jadi kami memunculkan dan mengeluarkan simbol dari tumpukan hingga kosong.



Tumpukan

$$abc * + de * f + g * +$$

Keluaran

Seperti sebelumnya, konversi ini hanya memerlukan $O(N)$ waktu dan bekerja dalam satu lintasan melalui input. Kita dapat menambahkan pengurangan dan pembagian ke dalam repertoar ini dengan menetapkan prioritas yang sama untuk pengurangan dan penjumlahan, serta prioritas yang sama untuk perkalian dan pembagian. Poin penting adalah bahwa ekspresi $a - b - c$ diubah menjadi $a - (b - c)$ dan tidak $a - b - c$. Algoritma kami melakukan hal yang benar, karena operator-operator ini berasosiasi dari kiri ke kanan. Ini tidak selalu terjadi secara umum, karena eksponensial mengasosiasikan kanan ke kiri: $2^3^2 = 2^{3^2} = 2^9 = 512$, bukan $(2^3)^2 = 8^2 = 64$. Kita tinggalkan soal penambahan eksponensial ke dalam daftar operator sebagai latihan.

Panggilan Fungsi

Algoritma untuk memeriksa simbol seimbang menyarankan cara untuk mengimplementasikan pemanggilan fungsi dalam bahasa prosedural dan berorientasi objek yang dikompilasi. Masalahnya di sini adalah ketika pemanggilan dilakukan ke fungsi baru, semua variabel lokal pada rutin pemanggil perlu disimpan oleh sistem, karena jika tidak, fungsi baru akan menimpa memori yang digunakan oleh variabel rutin pemanggil. Lebih lanjut, lokasi saat ini dalam rutin harus disimpan.

agar fungsi baru tahu ke mana harus pergi setelah selesai. Variabel-variabel umumnya telah ditetapkan oleh kompiler ke register mesin, dan pasti akan ada konflik (biasanya semua fungsi mendapatkan beberapa variabel yang ditetapkan ke register #1), terutama jika melibatkan rekursi. Alasan mengapa masalah ini mirip dengan penyeimbangan simbol adalah karena pemanggilan fungsi dan pengembalian fungsi pada dasarnya sama dengan tanda kurung buka dan tanda kurung tutup, sehingga ide yang sama seharusnya berfungsi.

Ketika suatu fungsi dipanggil, semua informasi penting yang perlu disimpan, seperti nilai register (yang berkaitan dengan nama variabel) dan alamat pengembalian (yang dapat diperoleh dari penghitung program, yang biasanya terdapat dalam sebuah register), disimpan "di selembar kertas" secara abstrak dan diletakkan di puncak tumpukan. Kemudian, kontrol ditransfer ke fungsi baru, yang bebas mengganti register dengan nilainya. Jika fungsi tersebut melakukan pemanggilan fungsi lain, prosedur yang sama akan dilakukan. Ketika fungsi ingin kembali, ia melihat "kertas" di puncak tumpukan dan memulihkan semua register. Kemudian, fungsi tersebut membuat lompatan pengembalian.

Jelas, semua pekerjaan ini dapat dilakukan menggunakan tumpukan, dan itulah yang terjadi di hampir setiap bahasa pemrograman yang menerapkan rekursi. Informasi yang disimpan disebut catatan aktivasi atau bingkai tumpukan. Biasanya, sedikit penyesuaian dilakukan: Lingkungan saat ini direpresentasikan di bagian atas tumpukan. Dengan demikian, pengembalian akan menampilkan lingkungan sebelumnya (tanpa menyalin). Tumpukan di komputer nyata sering kali bertambah dari batas atas partisi memori Anda ke bawah, dan pada banyak sistem tidak ada pemeriksaan luapan. Selalu ada kemungkinan Anda akan kehabisan ruang tumpukan karena memiliki terlalu banyak fungsi yang aktif secara bersamaan. Tak perlu dikatakan lagi, kehabisan ruang tumpukan selalu merupakan kesalahan fatal.

Dalam bahasa dan sistem yang tidak memeriksa stack overflow, program akan mengalami crash tanpa penjelasan yang jelas. Dalam keadaan normal, Anda seharusnya tidak kehabisan ruang stack; hal ini biasanya merupakan indikasi rekursi yang tak terkendali (lupa akan kasus dasar). Di sisi lain, beberapa program yang legal dan tampak tidak berbahaya dapat menyebabkan Anda kehabisan ruang stack. Rutin pada Gambar 1.25, yang mencetak sebuah kontainer, sepenuhnya legal dan sebenarnya benar. Rutin ini menangani kasus dasar kontainer kosong dengan benar, dan rekursinya baik-baik saja. Program ini dapat terbukti benar. Sayangnya, jika wadahnya

```

1  /**
2   * print container from start up to but not including end.
3   * /
4   template <typename Iterator>
5   void print( Iterator start, Iterator end, ostream & out = cout ){
6
7       if (start == end)
8           return;
9
10      out << *start++ << end;          // print and advance start
11      print( start, end, out );
12  }
```

Gambar 3.25 Penggunaan rekursi yang buruk: mencetak kontainer

```

1  /**
2   * print container from start up to but not including end .
3   * /
4   template <typename Iterator>
5   void print( Iterator start, Iterator end, ostream & out = cout ){
6
7       while (true)
8       {
9           if (start == end)
10              return;
11
12           out << *start++ << end;           // print and advance start
13       }
14  }
```

Gambar 3.26 Mencetak kontainer tanpa rekursi;sebuah compiler mungkin melakukan ini (Anda harus bukan)

Jika berisi 200.000 elemen untuk dicetak, akan ada tumpukan 200.000 rekaman aktivasi yang mewakili panggilan bersarang baris 11. Rekaman aktivasi biasanya berukuran besar karena banyaknya informasi yang dikandungnya, sehingga program ini kemungkinan akan kehabisan ruang tumpukan.(Jika 200.000 elemen tidak cukup untuk membuat program macet, ganti angka tersebut dengan angka yang lebih besar.)

Program ini adalah contoh penggunaan rekursi yang sangat buruk yang dikenal sebagai rekursi ekor. Rekursi ekor mengacu pada pemanggilan rekursif di baris terakhir. Rekursi ekor dapat dihilangkan secara mekanis dengan membungkus badan dalam ketika loop dan mengganti pemanggilan rekursif dengan satu penugasan per argumen fungsi. Ini mensimulasikan pemanggilan rekursif karena tidak ada yang perlu disimpan; setelah pemanggilan rekursif selesai, sebenarnya tidak perlu mengetahui nilai yang disimpan. Karena itu, kita bisa langsung menuju ke awal fungsi dengan nilai-nilai yang seharusnya digunakan dalam pemanggilan rekursif. Fungsi pada Gambar 3.26 menunjukkan versi yang telah ditingkatkan secara mekanis yang dihasilkan oleh algoritma ini. Penghapusan rekursi ekor sangat mudah sehingga beberapa kompiler melakukannya secara otomatis. Meskipun demikian, sebaiknya jangan sampai Anda tahu bahwa kompiler Anda tidak melakukannya.

Rekursi selalu dapat dihilangkan sepenuhnya (kompiler melakukannya dengan mengonversi ke bahasa assembly), tetapi melakukannya bisa sangat membosankan. Strategi umum ini memerlukan penggunaan tumpukan dan hanya bermanfaat jika Anda dapat menempatkan beban minimum pada tumpukan. Kita tidak akan membahas hal ini lebih lanjut, kecuali untuk menunjukkan bahwa meskipun program non-rekursif umumnya lebih cepat daripada program rekursif yang setara, keunggulan kecepatannya jarang membenarkan kurangnya kejelasan yang dihasilkan dari penghapusan rekursi.

3.7 Antrean ADT

Seperti tumpukan, antrian adalah daftar. Namun, dengan antrean, penyisipan dilakukan di satu sisi, sedangkan penghapusan dilakukan di sisi lainnya.

3.7.1 Model Antrean

Operasi dasar pada antrian adalah mengantrekan, yang menyisipkan elemen di akhir daftar (disebut belakang), dan mengeluarkan antrean, yang menghapus (dan mengembalikan) elemen di awal daftar (dikenal sebagai front). Gambar 1.2 menunjukkan model abstrak antrean.

3.7.2 Implementasi Array Antrean

Seperti halnya tumpukan, implementasi daftar apa pun legal untuk antrean. Seperti tumpukan, implementasi daftar tertaut dan array memberikan kecepatan yang cepat. $O(1)$ waktu berjalan untuk setiap operasi. Implementasi linked list cukup mudah dan dapat digunakan sebagai latihan. Sekarang kita akan membahas implementasi array untuk antrean.

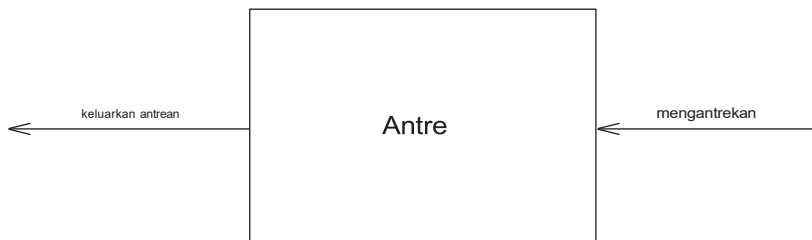
Untuk setiap struktur data antrian, kami menyimpan array, `Array`, dan posisi depan dan kembali, yang mewakili ujung antrean. Kami juga mencatat jumlah elemen yang sebenarnya ada dalam antrean, ukuran saat ini. Tabel berikut menunjukkan antrean dalam beberapa status peralihan.

			5	2	7	1		
			↑		↑			
			depan		kembali			

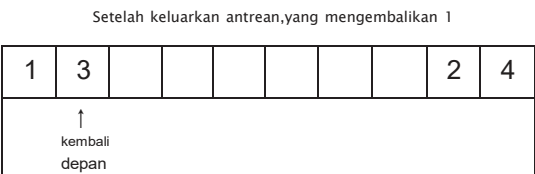
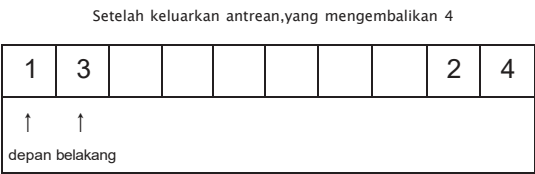
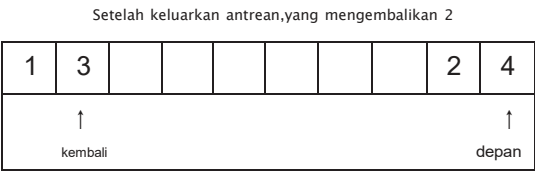
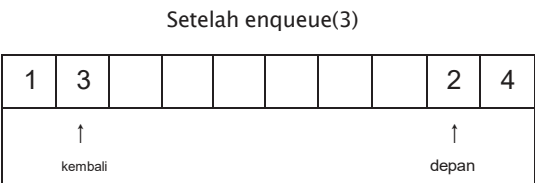
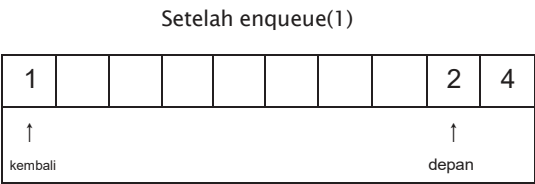
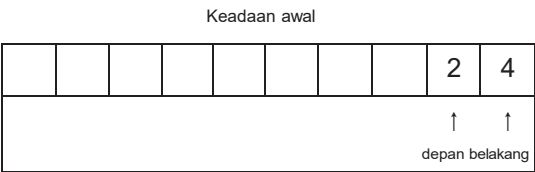
Operasinya harus jelas. Untuk mengantrekan sebuah elemen X , kami menambah ukuran saat ini dan kembali, lalu atur `theArray[kembali] = x`. Keluarkan antrean elemen, kami menetapkan nilai pengembalian ke `theArray[depan]`, penurunan ukuran saat ini, lalu menambah depan. Strategi lain mungkin dilakukan (akan dibahas nanti). Kami akan membahas pemeriksaan kesalahan nanti.

Ada satu masalah potensial dengan implementasi ini. Setelah 10 mengantrekan, antriannya nampaknya sudah penuh, karena kembali sekarang berada pada indeks array terakhir, dan selanjutnya mengantrekan akan berada di posisi tidak ada. Namun, mungkin hanya ada beberapa elemen dalam antrean, karena beberapa elemen mungkin telah dihapus dari antrean. Antrean, seperti tumpukan, seringkali tetap kecil meskipun terdapat banyak operasi.

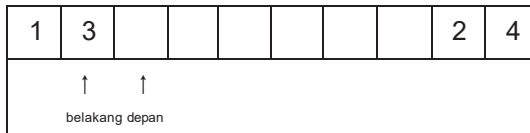
Solusi sederhananya adalah kapan pun depan atau kembali mencapai akhir array, ia akan dibungkus ke awal. Tabel berikut menunjukkan antrean selama beberapa operasi. Ini dikenal sebagai susunan melingkar pelaksanaan.



Gambar 3.27 Model antrian



Setelah dikeluarkan antrean, yang mengembalikan 3
dan membuat antrian kosong



Kode tambahan yang diperlukan untuk mengimplementasikan wraparound sangat minimal (meskipun mungkin menggandakan waktu proses). Jika menambahkan salah satu kembali atau depan menyebabkannya melewati array, nilainya diatur ulang ke posisi pertama dalam array.

Beberapa programmer menggunakan cara yang berbeda untuk merepresentasikan bagian depan dan belakang antrean. Misalnya, beberapa programmer tidak menggunakan entri untuk mencatat ukuran antrean, karena mereka bergantung pada kasus dasar bahwa ketika antrean kosong, $\text{back} = \text{front} - 1$. Ukuran dihitung secara implisit dengan membandingkan back dan front. Ini adalah cara yang sangat rumit, karena ada beberapa kasus khusus, jadi berhati-hatilah jika Anda perlu memodifikasi kode yang ditulis dengan cara ini. Jika ukuran saat ini tidak dipertahankan sebagai anggota data eksplisit, maka antriannya penuh ketika ada kapasitas $\text{Array}()$ -1 elemen, karena hanya kapasitas $\text{Array}()$ berbeda. Ukuran dapat dibedakan dan salah satunya adalah 0. Pilih gaya apa pun yang Anda suka dan pastikan semua rutin Anda konsisten. Karena ada beberapa opsi implementasi, mungkin ada baiknya menambahkan satu atau dua komentar dalam kode jika Anda tidak menggunakan ukuran saat ini anggota data.

Dalam aplikasi di mana Anda yakin bahwa jumlah mengantrekan tidak lebih besar dari kapasitas antrean, wraparound tidak diperlukan. Seperti halnya tumpukan, mengeluarkan antrean Operasi ini jarang dilakukan kecuali rutin pemanggil yakin bahwa antrean tidak kosong. Oleh karena itu, pemeriksaan kesalahan sering kali dilewati untuk operasi ini, kecuali dalam kode kritis. Hal ini umumnya tidak dapat dibenarkan, karena penghematan waktu yang mungkin Anda peroleh sangat minim.

3.7.3 Aplikasi Antrean

Ada banyak algoritma yang menggunakan antrean untuk menghasilkan waktu eksekusi yang efisien. Beberapa di antaranya terdapat dalam teori graf, dan kami akan membahasnya di Bab 9. Untuk saat ini, kami akan memberikan beberapa contoh sederhana penggunaan antrean.

Ketika pekerjaan dikirimkan ke percetakan, pekerjaan tersebut disusun berdasarkan urutan kedatangan. Jadi, pada dasarnya, pekerjaan yang dikirim ke percetakan ditempatkan dalam antrean.

Hampir setiap antrean di dunia nyata (seharusnya) merupakan antrean. Misalnya, antrean di loket tiket adalah antrean, karena layanannya berdasarkan siapa cepat dia dapat.

Contoh lain menyangkut jaringan komputer. Ada banyak pengaturan jaringan komputer pribadi mana disk terhubung ke satu mesin, yang dikenal sebagai file server. Pengguna pada mesin lain diberi akses ke berkas berdasarkan siapa yang datang pertama akan dilayani pertama, jadi struktur datanya adalah antrean.

Kami mengatakan pada dasarnya karena pekerjaan bisa dihentikan. Ini sama saja dengan penghapusan dari tengah antrean, yang merupakan pelanggaran terhadap definisi yang ketat.

Contoh lebih lanjut termasuk yang berikut ini:

- Panggilan ke perusahaan besar umumnya dimasukkan ke antrean ketika semua operator sedang sibuk.
- Di universitas besar dengan sumber daya terbatas, mahasiswa harus menandatangani daftar tunggu jika semua komputer terisi. Mahasiswa yang paling lama menggunakan komputer akan dipaksa keluar terlebih dahulu, dan mahasiswa yang paling lama menunggu akan diizinkan masuk berikutnya.

Seluruh cabang matematika yang dikenal sebagai teori antrian Berkaitan dengan komputasi, secara probabilistik, berapa lama pengguna diperkirakan akan menunggu dalam suatu antrean, seberapa panjang antrean tersebut, dan pertanyaan-pertanyaan serupa lainnya. Jawabannya bergantung pada seberapa sering pengguna tiba di antrean dan berapa lama waktu yang dibutuhkan untuk memproses pengguna setelah pengguna dilayani. Kedua parameter ini diberikan sebagai fungsi distribusi probabilitas. Dalam kasus sederhana, jawaban dapat dihitung secara analitis. Contoh kasus sederhana adalah saluran telepon dengan satu operator. Jika operator sibuk, penelepon akan ditempatkan dalam antrean tunggu (hingga batas maksimum tertentu). Masalah ini penting bagi bisnis, karena penelitian menunjukkan bahwa orang-orang cenderung cepat menutup telepon.

Jika ada koperator, maka masalah ini jauh lebih sulit dipecahkan. Masalah yang sulit dipecahkan secara analitis seringkali diselesaikan dengan simulasi. Dalam kasus kami, kami perlu menggunakan antrean untuk melakukan simulasi. Jika besar, kita juga membutuhkan struktur data lain untuk melakukan ini secara efisien. Kita akan melihat cara melakukan simulasi ini di Bab 6. Kita kemudian dapat menjalankan simulasi untuk beberapa nilai k yang memberikan waktu tunggu yang wajar.

Penggunaan tambahan untuk antrean berlimpah, dan seperti halnya tumpukan, sungguh mengejutkan bahwa struktur data yang begitu sederhana bisa menjadi begitu penting.

Ringkasan

Bab ini menjelaskan konsep ADT dan mengilustrasikannya dengan tiga tipe data abstrak yang paling umum. Tujuan utamanya adalah memisahkan implementasi ADT dari fungsinya. Program harus mengetahui apa yang dilakukan oleh operasi-operasi tersebut, tetapi sebenarnya lebih baik jika tidak mengetahui bagaimana cara kerjanya.

List, stack, dan queue mungkin merupakan tiga struktur data fundamental dalam seluruh ilmu komputer, dan penggunaannya di dokumentasikan melalui berbagai contoh. Secara khusus, kita melihat bagaimana stack digunakan untuk melacak pemanggilan fungsi dan bagaimana rekursi sebenarnya diimplementasikan. Hal ini penting untuk dipahami, bukan hanya karena memungkinkan penggunaan bahasa prosedural, tetapi karena mengetahui bagaimana rekursi diimplementasikan akan menghilangkan banyak misteri yang menyelimuti penggunaannya. Meskipun rekursi sangat ampuh, ia bukanlah operasi yang sepenuhnya bebas; penyalahgunaan rekursi dapat mengakibatkan program mengalami crash.

Latihan

- 3.1 Anda diberikan sebuah daftar, L , dan daftar lainnya, P , berisi bilangan bulat yang diurutkan dalam urutan menaik. Operasi $\text{print}(L, P)$ akan mencetak elemen dalam L yang berada pada posisi yang ditentukan oleh P . Misalnya, jika $P = 1, 3, 4, 6$, unsur-unsur pada posisi 1, 3, 4, dan 6 di L dicetak. Tuliskan prosedurnya $\text{print}(L, P)$. Anda hanya dapat menggunakan operasi kontainer STL publik. Berapa lama waktu berjalan prosedur Anda?

- 1.2 Tukar dua elemen yang berdekatan dengan hanya menyesuaikan tautan (dan bukan data)
 - a. menggunakan daftar tertaut tunggal
 - b. daftar tertaut ganda
- 1.3 Terapkan STL find rutin yang mengembalikan pengulangan berisi kemunculan pertama X dalam kisaran yang dimulai pada awal dan meluas hingga tetapi tidak termasuk akhir. Jika X tidak ditemukan, akhir dikembalikan. Ini adalah nonkelas (fungsi global) dengan tanda tangan


```

      templat <typename Iterator, typename Object>
      iterator find (Iterator start, Iterator end, const Object & x );
      
```
- 1.4 Diberikan dua daftar yang diurutkan, $L1$ dan $L2$, tulis prosedur untuk menghitung $L1 \cap L2$ hanya menggunakan operasi daftar dasar.
- 1.5 Diberikan dua daftar yang diurutkan, $L1$ dan $L2$, tulis prosedur untuk menghitung $L1 \cup L2$ hanya menggunakan operasi daftar dasar.
- 1.6 Itu Masalah Josephus adalah permainan berikut ini: Norang, bernomor 1 sampai N, duduk melingkar. Dimulai dari orang 1, kentang panas diedarkan. Setelah M berlalu, orang yang memegang kentang panas tereliminasi, lingkaran menutup barisan, dan permainan berlanjut dengan orang yang duduk setelah orang yang tereliminasi mengambil kentang panas. Orang terakhir yang tersisa menang. Jadi, jika $M=0$ dan $N=5$, pemain dieliminasi secara berurutan, dan pemain 5 menang. Jika $M=1$ dan $N=5$, urutan eliminasinya adalah 2, 4, 1, 5.
 - a. Tulislah sebuah program untuk menyelesaikan masalah Josephus untuk nilai-nilai umum M dan N Usahakan program Anda seefisien mungkin. Pastikan Anda membuang sel.
 - b. Berapa lama waktu berjalan program Anda?
 - c. Jika $M=1$. Berapa lama waktu berjalan program Anda? Bagaimana kecepatan aktual dipengaruhi oleh menghapus rutin untuk nilai besar N ($N > 100.000$)?
- 1.7 Ubah Vektor kelas untuk menambahkan pemeriksaan batas untuk pengeindeksan.
- 1.8 Menambahkan menyisipkan dan menghapus ke Vektor kelas.
- 1.9 Menurut standar C++, untuk vektor, panggilan untuk dorong back, pop_kembali, menyisipkan, atau menghapus membatalkan (berpotensi membuat basi) semua iterator yang melihat vektor. Mengapa?
- 1.10 Ubah Vektor kelas untuk menyediakan pemeriksaan iterator yang ketat dengan menjadikan iterator sebagai tipe kelas, alih-alih variabel penunjuk. Bagian tersulit adalah menangani iterator yang basi, seperti yang dijelaskan dalam Latihan 1.9.
- 1.11 Asumsikan bahwa sebuah daftar tertaut tunggal diimplementasikan dengan simpul header, tetapi dan hanya menyimpan penunjuk ke simpul header. Tulis sebuah kelas yang menyertakan metode untuk
 - a. mengembalikan ukuran linked list
 - b. mencetak daftar tertaut
 - c. uji apakah suatu nilai X terdapat dalam daftar tertaut
 - d. menambahkan nilai X jika belum ada dalam daftar tertaut
 - e. menghapus suatu nilai X jika terdapat dalam linked list
- 1.12 Ulangi Latihan 1.11, pertahankan daftar tertaut tunggal dalam urutan terurut.
- 1.13 Tambahkan dukungan untuk operator-ke Daftar kelas iterator.

- 1.14 Melihat ke depan dalam iterator STL memerlukan penerapan operator++, yang pada gilirannya memajukan iterator. Dalam beberapa kasus, melihat item berikutnya dalam daftar, tanpa maju ke sana, mungkin lebih baik. Tulis fungsi anggota dengan deklarasi

```
const_iterator operator + ( int k ) const;
```

untuk memfasilitasi hal ini dalam kasus umum. Biner operator+ mengembalikan iterator yang sesuai dengan k posisi di depan saat ini.

- 1.15 Tambahkan sambatan operasi keDaftar kelas. Deklarasi metode void

```
splice( posisi iterator, List<T> & lst );
```

menghapus semua item dariterakhir, menempatkannya sebelum posisi di dalam Daftar *ini. Dan*ini Harus ada daftar yang berbeda. Rutinitas Anda harus berjalan dalam waktu yang konstan.

- 1.16 Tambahkan iterator terbalik ke STLDaftarimplementasi kelas. Definiskaniterator_terbalik Dan const_reverse_iteratorTambahkan metode mulai Dan membelah untuk mengembalikan iterator terbalik yang sesuai yang mewakili posisi sebelum endmarker dan posisi yang merupakan node header. Iterator terbalik secara internal membalikkan makna dari++Dan--operator. Anda seharusnya dapat mencetak daftar L secara terbalik dengan menggunakan kode

```
list<Object>::reverse_iterator itr = L.rbegin();
sementara( itr != L.rend()
) cout << *itr++ << endl;
```

- 1.17 Ubah Daftar kelas untuk menyediakan pemeriksaan iterator yang ketat dengan menggunakan ide-ide yang disarankan di akhir Bagian 3.5.
- 1.18 Ketika sebuah metode menghapus diterapkan pada daftar, hal tersebut membatalkan semua pengulangan yang merujuk pada node yang dihapus. Iterator seperti itu disebut basi. Jelaskan algoritma efisien yang menjamin bahwa setiap operasi pada iterator basi akan bertindak seolah-olah iterator tersebut saat ini bernilai nullptr. Perhatikan bahwa mungkin terdapat banyak iterator yang sudah usang, dan Anda harus menjelaskan kelas mana yang perlu ditulis ulang agar algoritma tersebut dapat diimplementasikan.
- 1.19 Tulis ulang Daftar kelas tanpa menggunakan node header dan tail dan menjelaskan perbedaan antara kelas tersebut dan kelas yang disediakan di Bagian 1.5.
- 1.20 Alternatif untuk strategi penghapusan yang kami berikan adalah dengan menggunakan penghapusan
- 1.21 malas. Untuk menghapus suatu elemen, kita cukup menandainya sebagai terhapus (menggunakan bidang bit tambahan). Jumlah elemen yang dihapus dan tidak dihapus dalam daftar tetap tercatat sebagai bagian dari struktur data. Jika jumlah elemen yang dihapus sama dengan jumlah elemen yang tidak dihapus, kita menelusuri seluruh daftar, menjalankan algoritma penghapusan standar pada semua simpul yang ditandai.

a. Sebutkan keuntungan dan kerugian dari penghapusan malas.

b. Tulis rutin untuk mengimplementasikan operasi daftar tertaut standar menggunakan penghapusan malas.

- 1.22 Tulis program untuk memeriksa simbol keseimbangan dalam bahasa berikut:

a. Pascal (begin/end,(),[],{}).

b. C++ ((* */(),[],{}).

c. Jelaskan cara mencetak pesan kesalahan yang mungkin mencerminkan kemungkinan penyebabnya.

- 1.23 Tulis program untuk mengevaluasi ekspresi postfix.
- 1.24 a. Tulislah sebuah program untuk mengkonversikan ekspresi infiks yang mengandung $(,), +, -, *$, Dan/ untuk postfix.
 b. Tambahkan operator eksponensial ke repertoar Anda.
 c. Tulis program untuk mengubah ekspresi postfix menjadi infix.
- 1.25 Tulis rutin untuk mengimplementasikan dua tumpukan hanya menggunakan satu larik. Rutinitas tumpukan Anda tidak boleh mendeklarasikan luapan kecuali setiap slot dalam larik digunakan.
- a. Usulkan struktur data yang mendukung tumpukan dorongan Dan pop operasi dan operasi ketiga temukan Min, yang mengembalikan elemen terkecil dalam struktur data, semuanya dalam $O(1)$ waktu terburuk.
 b. Buktikan jika kita menambahkan operasi keempat hapus Min yang menemukan dan menghilangkan elemen terkecil, maka setidaknya salah satu operasi harus mengambil $(\log N)$ waktu. (Ini memerlukan membaca Bab 7.)
- 1.26 Tunjukkan cara mengimplementasikan tiga tumpukan dalam satu susunan.
- 1.27 Jika rutinitas rekursif di Bagian 2.4 yang digunakan untuk menghitung angka Fibonacci dijalankan untuk $N=50$, apakah ruang tumpukan kemungkinan akan habis? Mengapa atau mengapa tidak?
- 1.28 Adeque adalah struktur data yang terdiri dari daftar item yang memungkinkan dilakukannya operasi berikut:
 dorong(x): Masukkan item X di bagian depan deque.
 Pop () : Keluarkan item depan dari deque dan kembalikan.
 Menyuntikkan (x): Masukkan item X di bagian belakang deque.
 Mengeluarkan () : Keluarkan item belakang dari deque dan kembalikan. Tulis rutinitas untuk mendukung deque yang mengambil $O(1)$ waktu per operasi.
- 1.29 Tulislah algoritma untuk mencetak daftar tertaut tunggal secara terbalik, hanya menggunakan spasi tambahan yang konstan. Instruksi ini menyiratkan bahwa Anda tidak dapat menggunakan rekursi, tetapi Anda dapat mengasumsikan bahwa algoritma Anda adalah fungsi anggota daftar. Dapatkah algoritma semacam itu ditulis jika rutinnya adalah fungsi anggota konstan?
- 1.30 a. Tuliskan implementasi array dari daftar yang dapat menyesuaikan diri. Dalam sebuah daftar penyesuaian diri, semua penyisipan dilakukan di bagian depan. Daftar penyesuaian sendiri menambahkan menemukan operasi, dan ketika suatu elemen diakses oleh menemukan, item tersebut dipindahkan ke bagian depan daftar tanpa mengubah urutan relatif item lainnya.
 b. Tulis implementasi linked list dari self-adjusting list.
 c. Misalkan setiap elemen memiliki probabilitas tetap, P_i , untuk diakses. Tunjukkan bahwa elemen dengan probabilitas akses tertinggi diperkirakan berada di dekat bagian depan.
- 1.31 Terapkan kelas tumpukan secara efisien menggunakan daftar tertaut tunggal, tanpa simpul header atau tail.
- 1.32 Terapkan kelas antrian secara efisien menggunakan daftar tertaut tunggal, tanpa simpul tajuk atau ekor.
- 1.33 Implementasikan kelas antrian secara efisien menggunakan array melingkar. Anda dapat menggunakan vektor (daripada array primitif) sebagai struktur array yang mendasarinya.
- 1.34 Daftar tertaut berisi siklus jika, dimulai dari beberapa simpul P , setelah jumlah yang cukup Berikutnya tautan membawa kita kembali ke simpul P . P tidak harus menjadi node pertama

dalam daftar. Asumsikan Anda diberi daftar tertaut yang berisi N simpul; namun, nilai N tidak diketahui.

a. Mendesain sebuah $O(N)$ untuk menentukan apakah daftar tersebut berisi siklus. Anda dapat menggunakan HAI(

N) ruang ekstra.

b. Ulangi bagian (a), tetapi gunakan hanya $O(1)$ ruang tambahan. (Petunjuk:Gunakan dua iterator yang awalnya berada di awal daftar tetapi maju dengan kecepatan berbeda.)

a. f

1.35 Misalkan kita memiliki pointer ke sebuah node dalam daftar tertaut tunggal yang dijamin menjadi simpul terakhir dalam daftar. Kami tidak memiliki petunjuk ke node lain (kecuali dengan mengikuti tautan). Jelaskan $O(1)$ algoritma yang secara logis menghapus nilai yang disimpan dalam node tersebut dari daftar tertaut, dengan tetap menjaga integritas daftar tertaut. Petunjuk: Libatkan simpul berikutnya.)

1.36 Misalkan sebuah daftar tertaut tunggal diimplementasikan dengan header dan simpul ekor. Jelaskan algoritma waktu konstan untuk

a. masukkan item X sebelum posisi P (diberikan oleh sebuah iterator)

b. keluarkan barang yang disimpan pada posisinya P (diberikan oleh sebuah iterator)